11: Caches II

Engr 315: Hardware / Software Codesign

Andrew Lukefahr
Indiana University

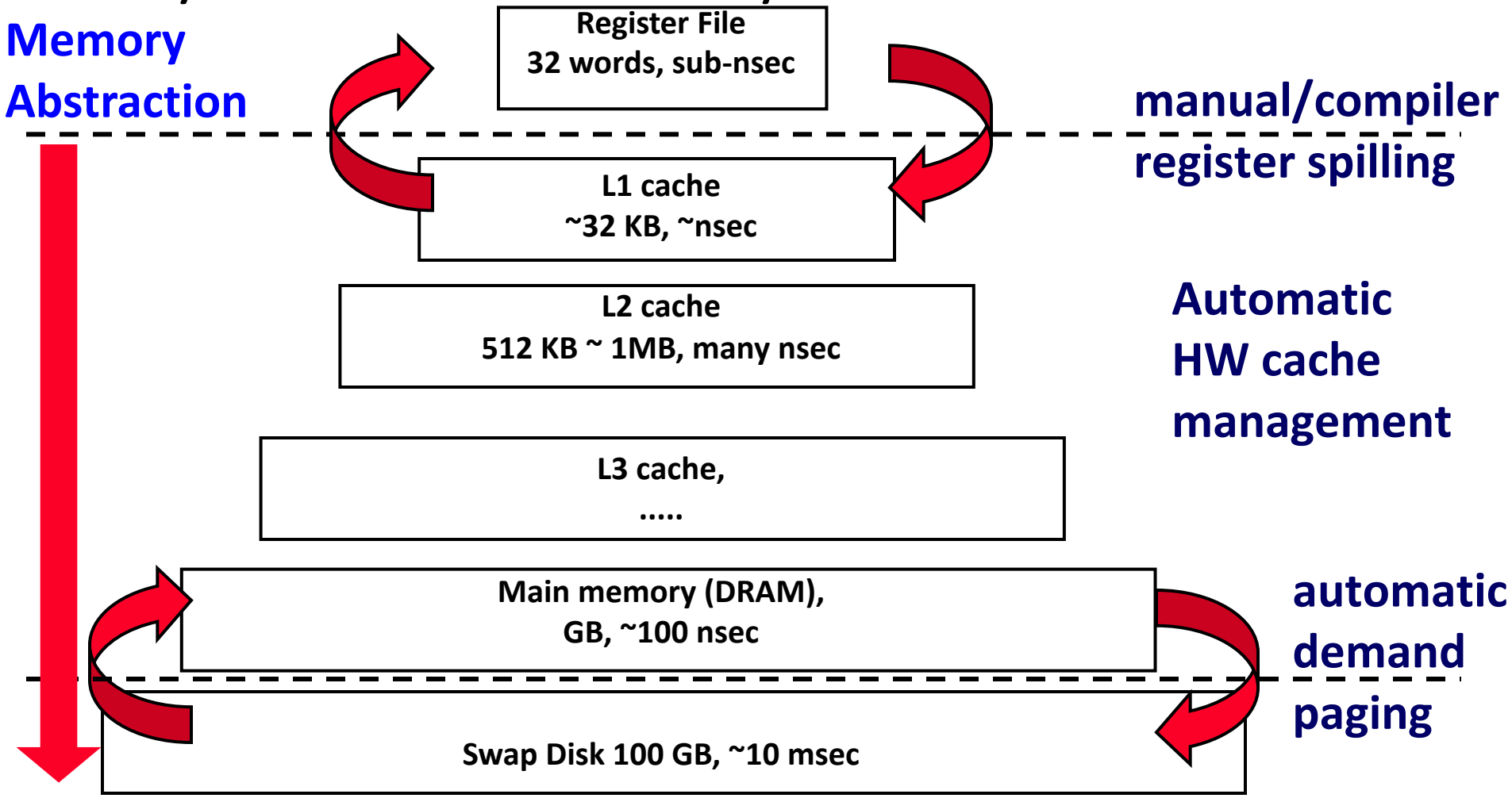
Some material taken from EECS370 at U. of Michigan

The Projects

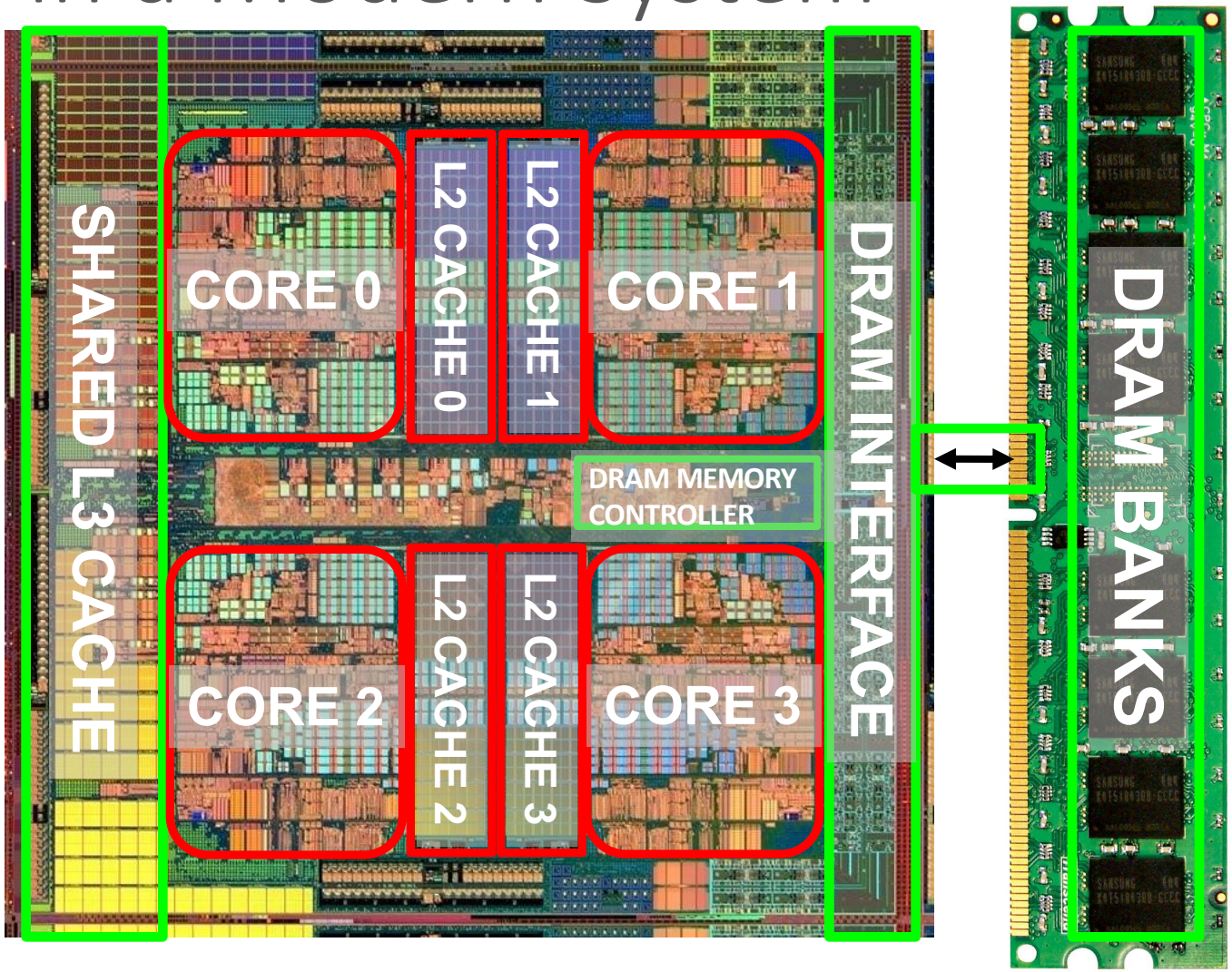
- P3: should be due
- P4: Count the number of binary 1's in a file.
  - Going to involve Linux on your Pynq.
  - I still need to figure out *how* to do that.

*Exam is coming up

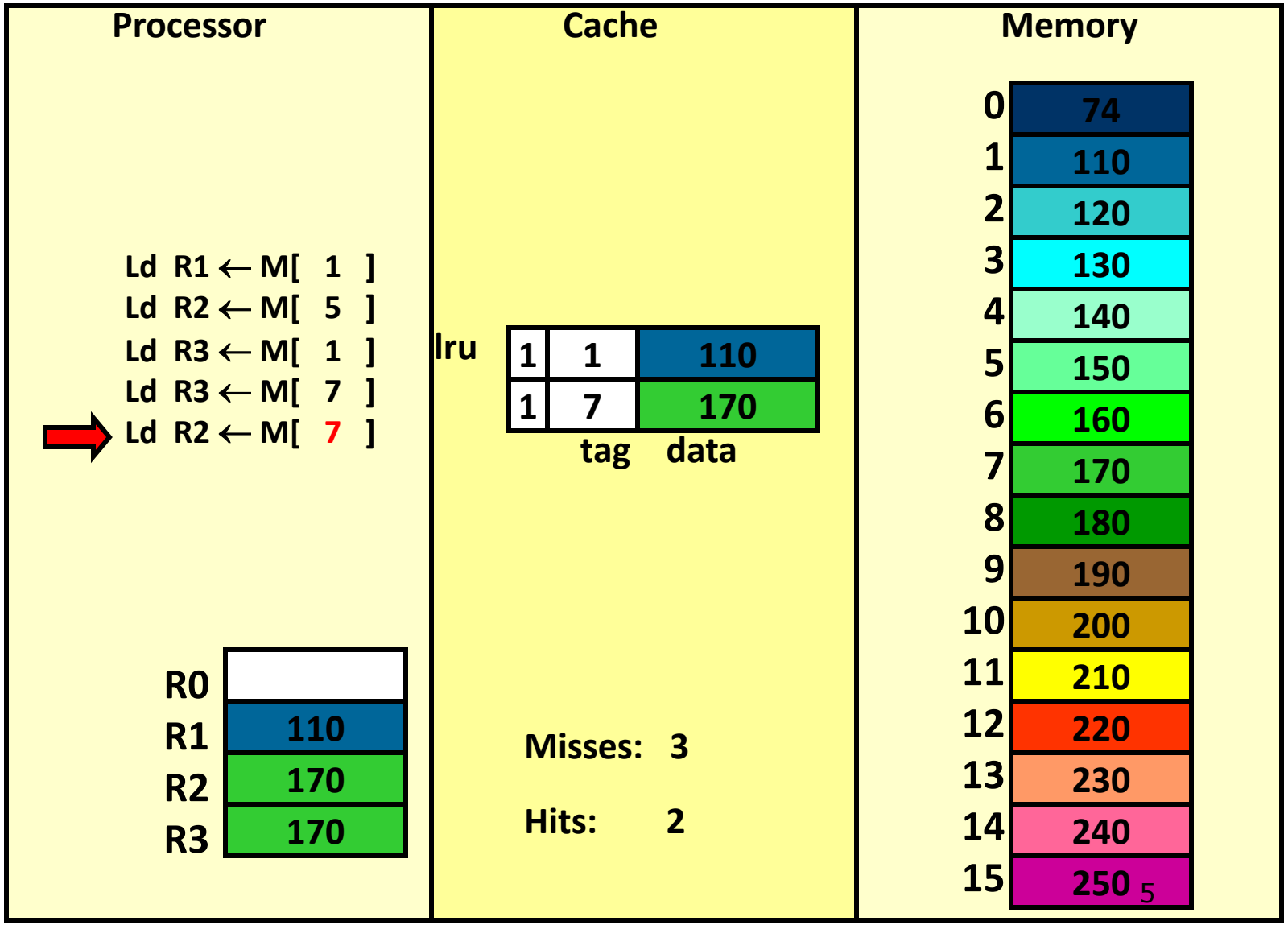
Memory in a Modern System



Memory in a Modern System



Last Time: A Very Simple Memory System

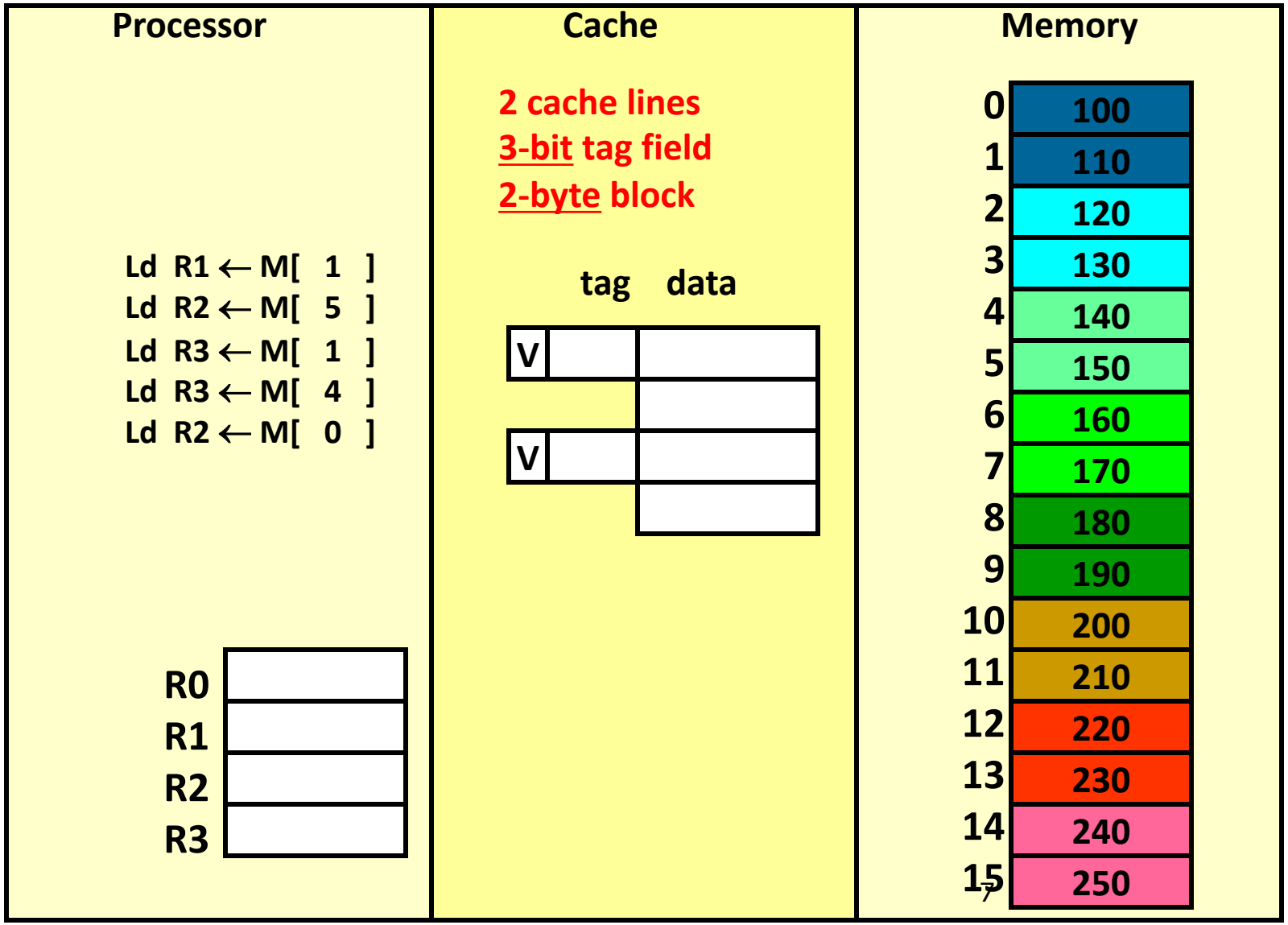


Can we reduce the overhead for each cache line?

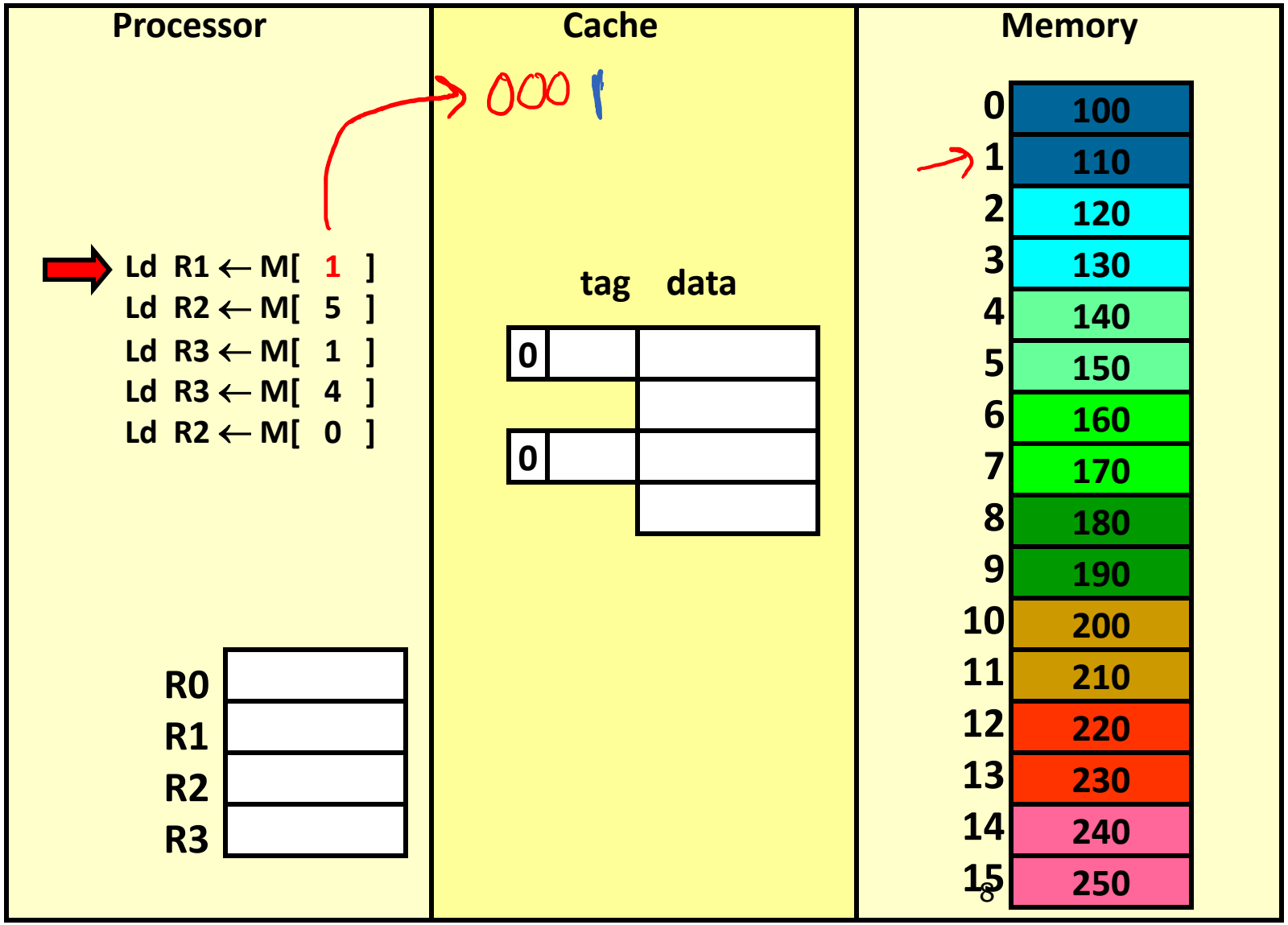
lru			
	1	1	110
	1	7	170
		tag	data

- ❑ Have a small address.
  - Impractical, and caches are supposed to be micro-architectural
- ❑ Cache bigger units than bytes
  - Each block has a single tag, and blocks can be whatever size we choose

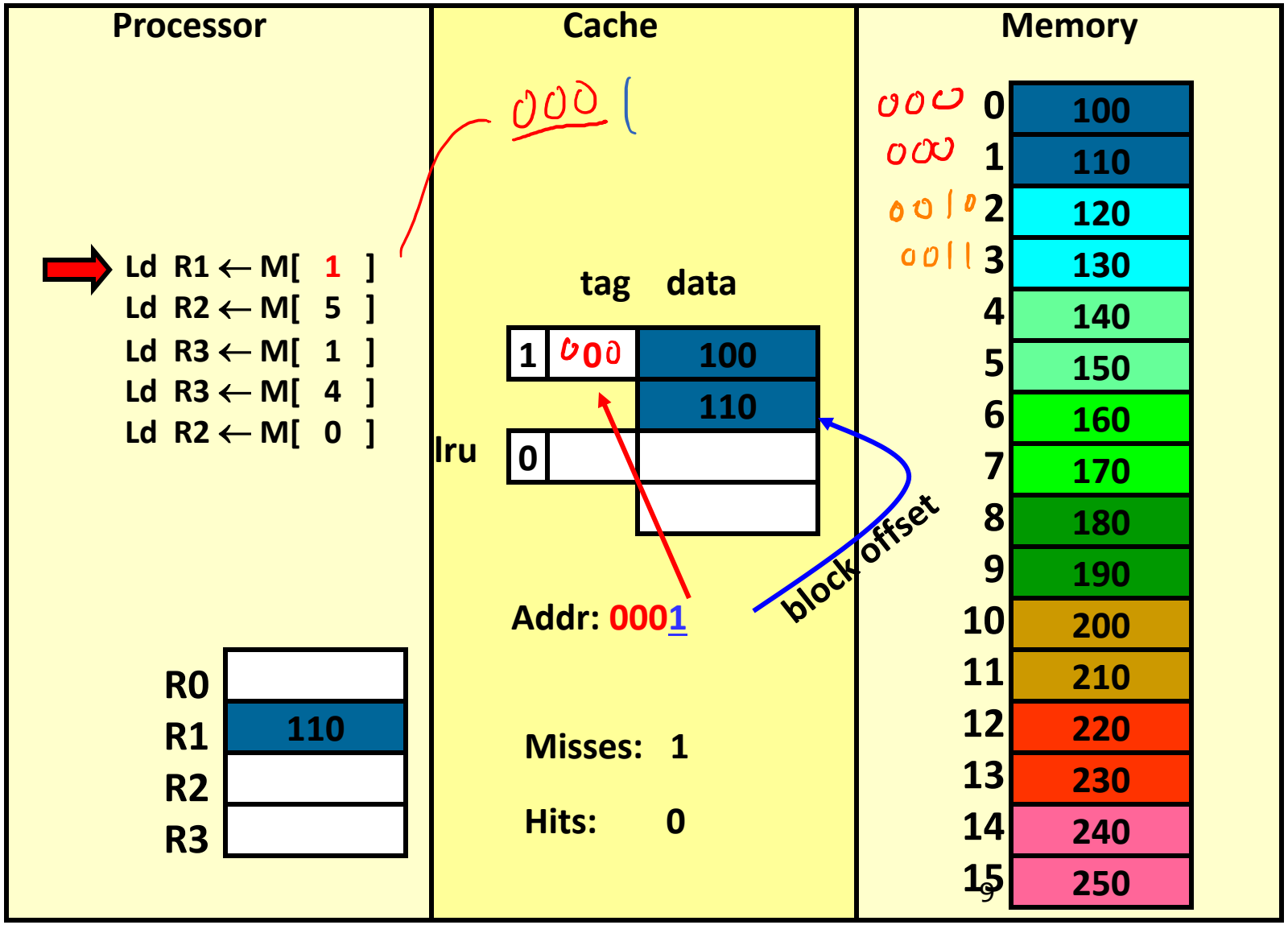
Block size for caches



Block size for caches



Block size for caches



Block size for caches

Processor

Ld R1 ← M[1]
Ld R2 ← M[5]
Ld R3 ← M[1]
Ld R3 ← M[4]
Ld R2 ← M[0]

R0	
R1	110
R2	
R3	

Cache

	tag	data
1	0	100
		110
lru 0	2	140
		150

Misses: 1

Hits: 0

Memory

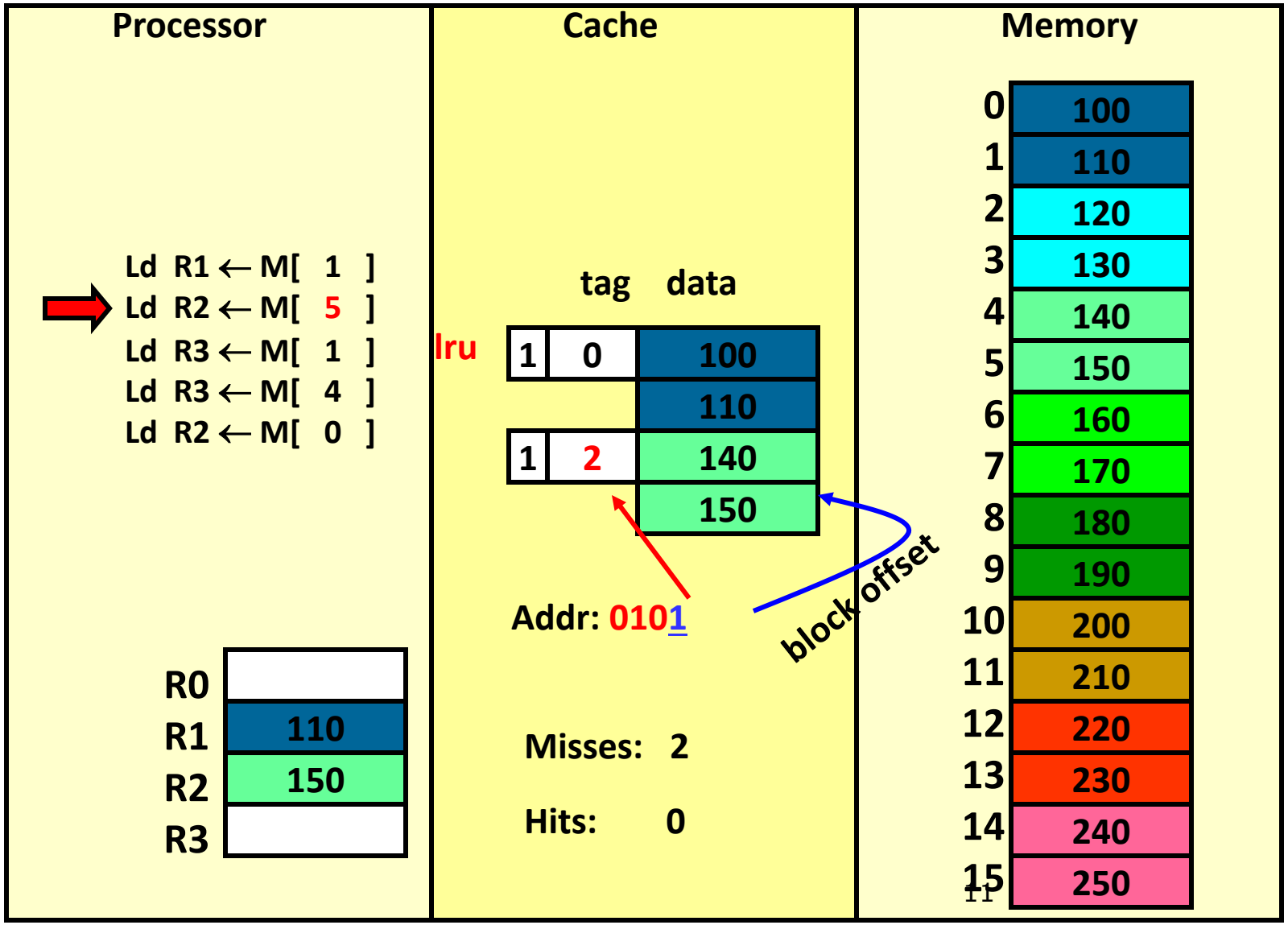
	blk	
0	0	100
1		110
2	1	120
3		130
4	2	140
5		150
6		160
7		170
8		180
9		190
10		200
11		210
12		220
13		230
14		240
15		250

000 0
000 1

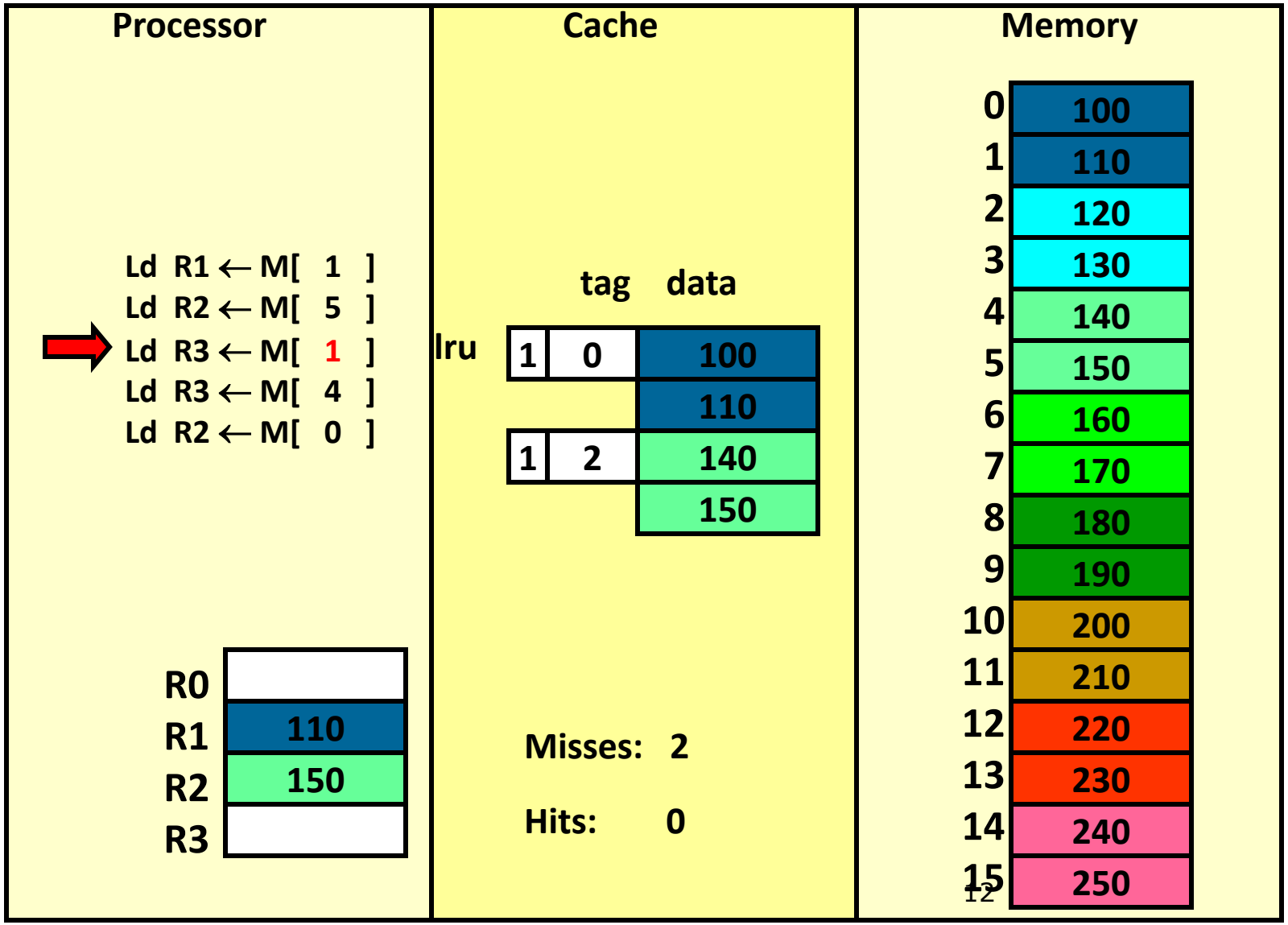
001 0
001 1

010 0
010 1

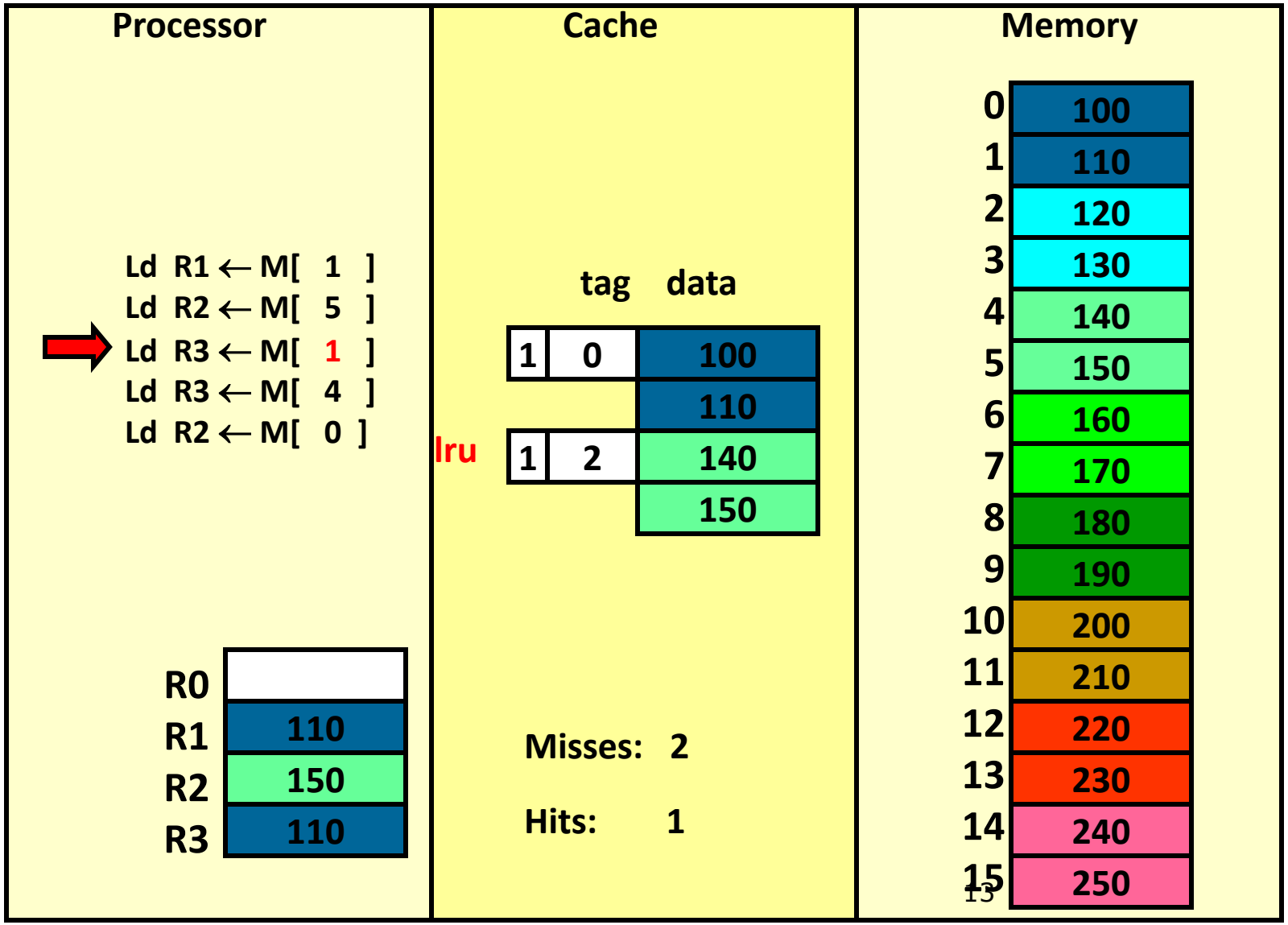
Block size for caches



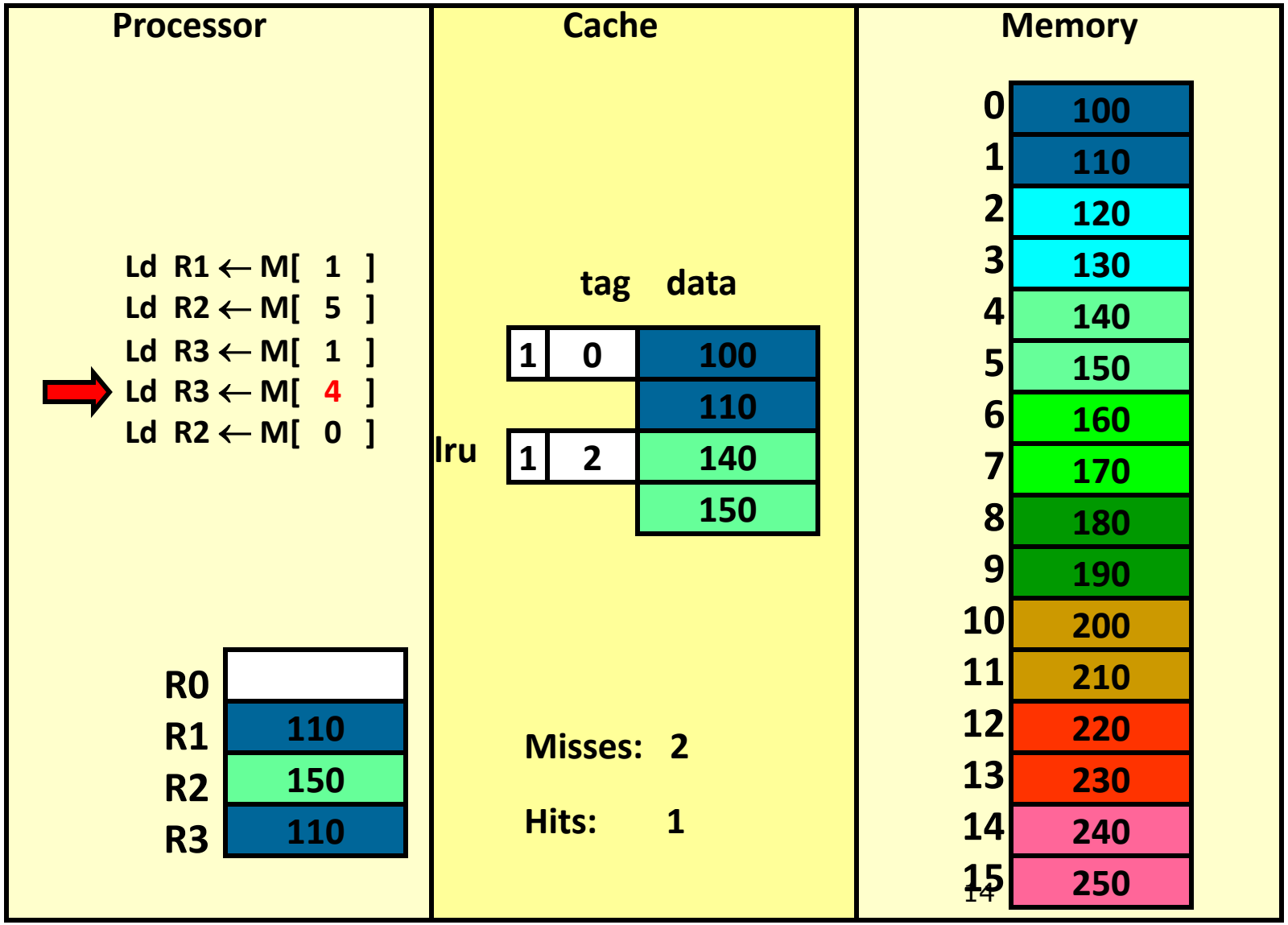
Block size for caches



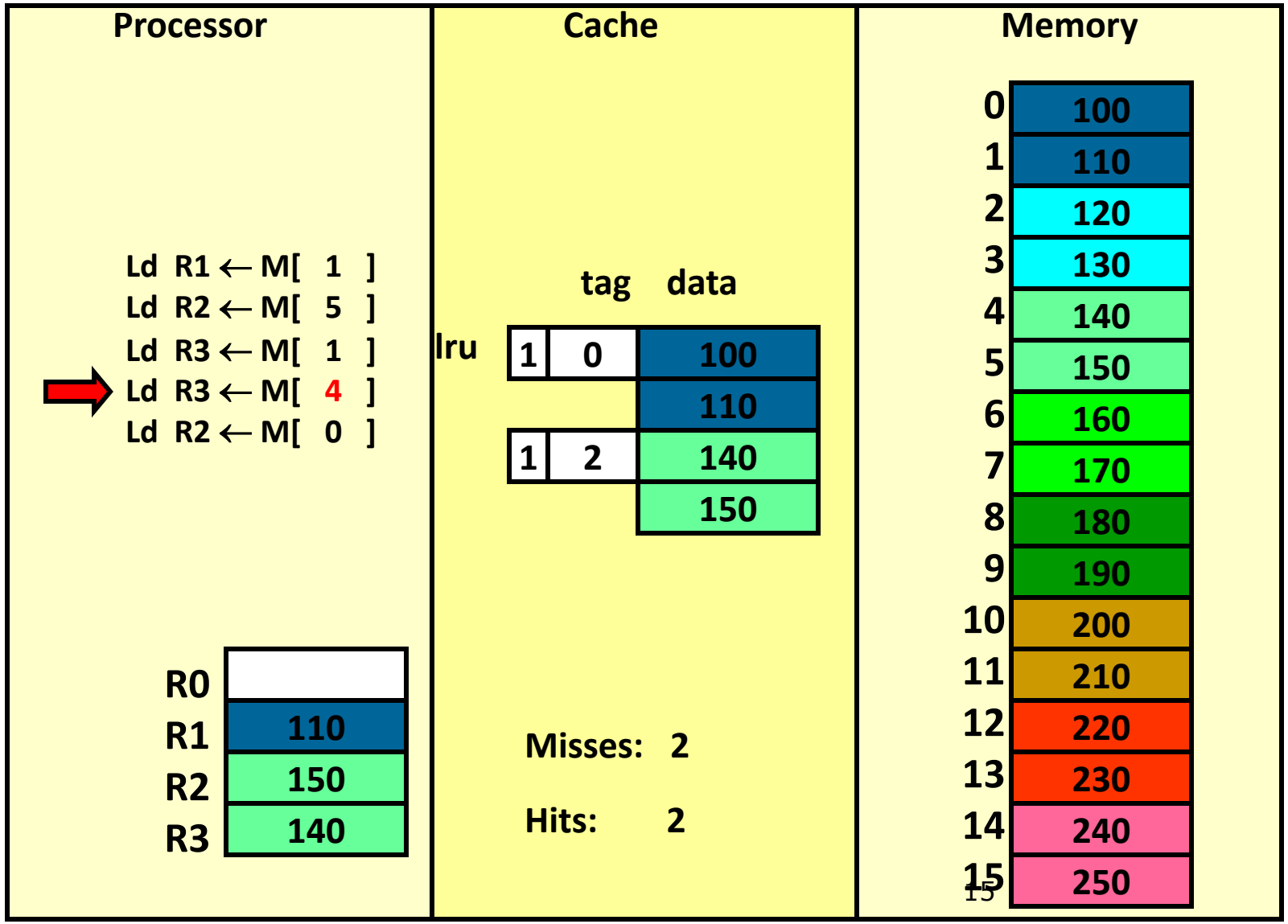
Block size for caches



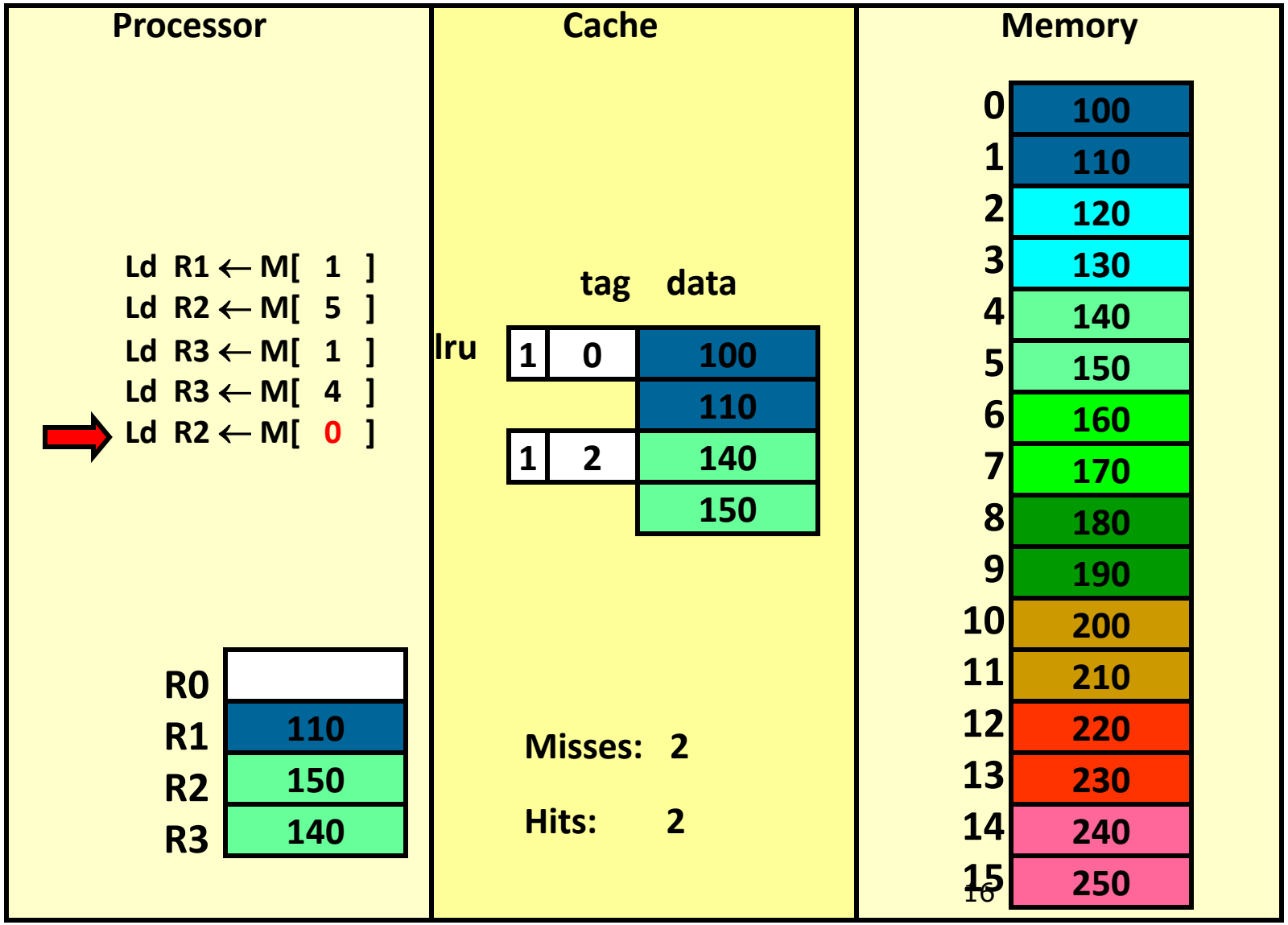
Block size for caches



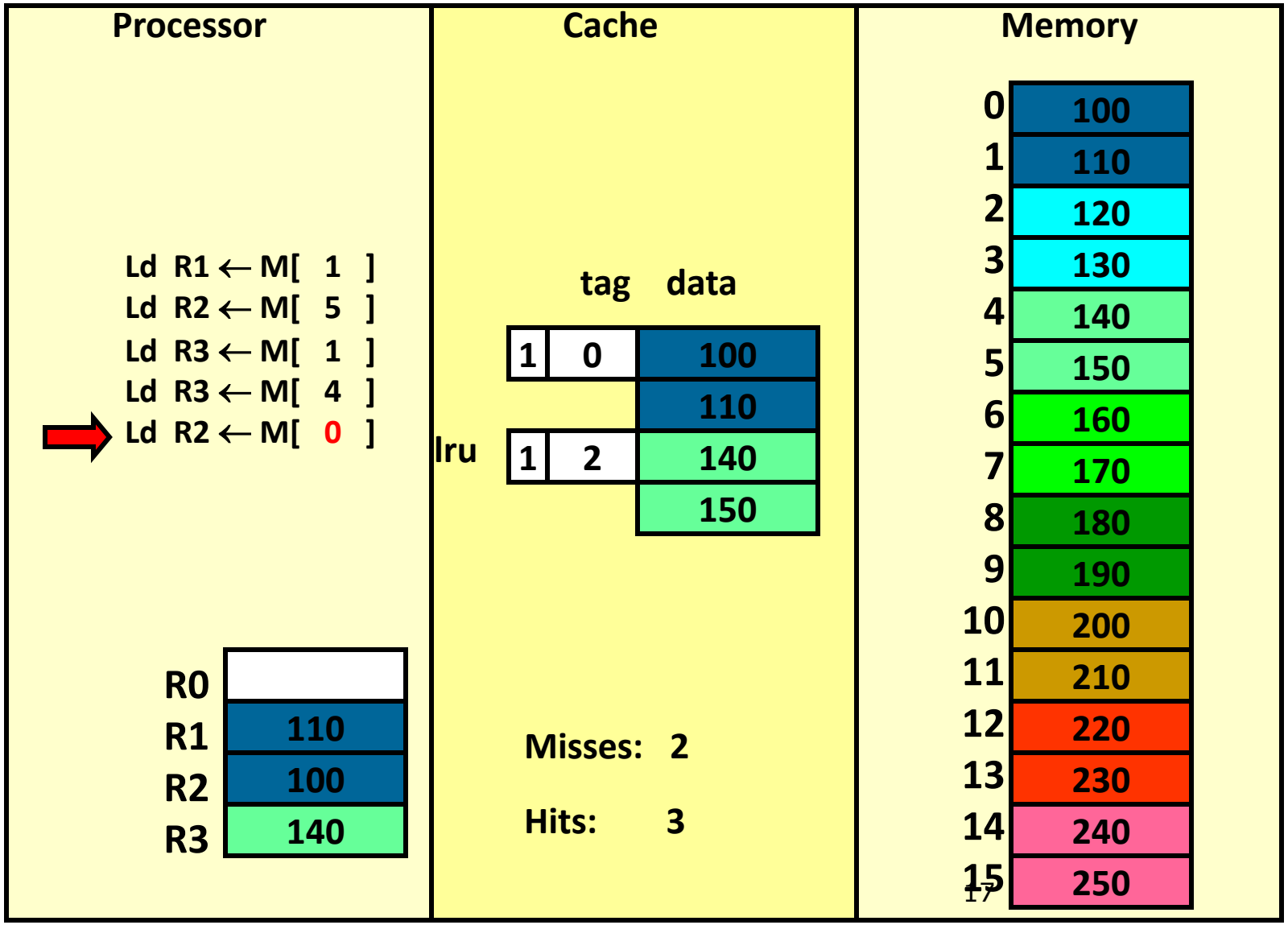
Block size for caches



Block size for caches



Block size for caches

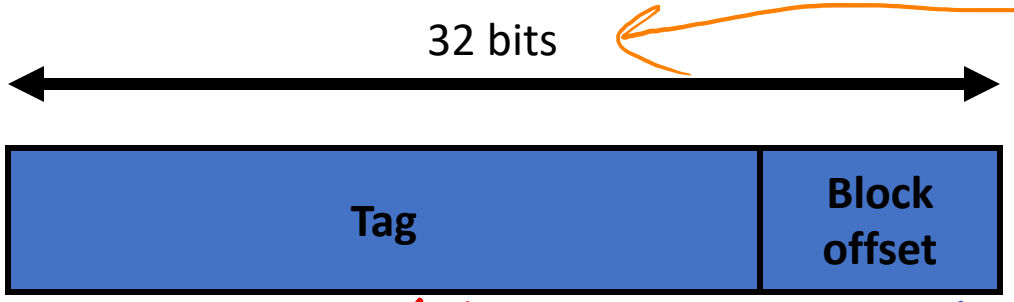


Spatial Locality

- Notice that when we accessed address 1, we also brought in address 0
  - This turned out to be a good thing since we later referenced address 0 and found it in the cache

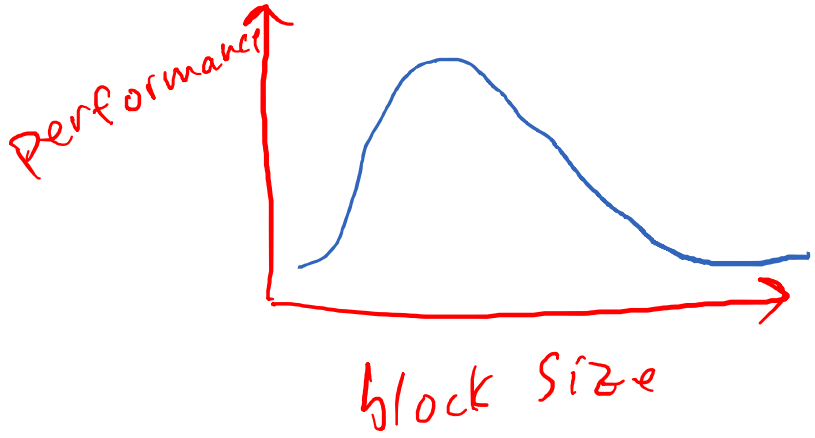
- Spatial locality: if we reference a memory location (e.g., 1000), we are likely to reference a location near it (e.g. 1001)

How does block offset affect tag size?



	Tag	Block offset
	XXX	XX
• 1 Byte Blocks:	32-bits	0 bits
• 2 Byte Blocks:	31-bits	1 bit
• 4 Byte Blocks:	30-bits	2 bits
• 32 Byte Blocks:	27-bits +	5 bits = 32

Basic Cache Organization



- Decide on the block size
  - How? Simulate lots of different block sizes and see which one gives the best performance
  - Most systems use a block size between 32 bytes and 128 bytes
  - Longer sizes reduce the overhead by:
    - Reducing the number of CAM entries
    - Reducing the size of each CAM entry

Practice Problem– Compute the register and cache state after executing the instruction sequence

Processor

miss ✓ Ld R0 ← M[3]
miss ✓ Ld R2 ← M[12]
hit → ✓ Ld R3 ← M[15]
miss → Ld R1 ← M[4]
miss → Ld R2 ← M[9]

R0	
R1	
R2	
R3	

Cache

2 cache lines
2-bit tag field
4-byte block

V (Valid)	tag	data
1	01	71
		150
		162
		172
1	(10)	18
		21
		33
		28

LRU

Memory

0	78
1	29
2	120
3	123
4	71
5	150
6	162
7	173
8	18
9	21
10	33
11	28
12	19
13	200
14	210
15	225

00 00
00 01
00 10
00 11
01 00
01 01
01 10
01 11
10 00
10 01
10 10
10 11
11 00
11 01
11 10
11 11

Solution to Practice Problem

Ld R0 ← M[3]

V	tag	data
1	0	78
		29
		120
		123
0		

lru

miss

Ld R2 ← M[12]

V	tag	data
1	0	78
		29
		120
		123
1	3	19
		200
		210
		225

lru

miss

Ld R3 ← M[15]

V	tag	data
1	0	78
		29
		120
		123
1	3	19
		200
		210
		225

lru

hit

Ld R1 ← M[4]

V	tag	data
1	1	71
		150
		162
		173
1	3	19
		200
		210
		225

lru

miss

Ld R2 ← M[9]

V	tag	data
1	1	71
		150
		162
		173
1	2	18
		21
		33
		28

lru

miss

Class Problem 1

- Given a cache with the following configuration: total size is 8 bytes, block size is 2 bytes, *fully associative*, LRU replacement. The memory address size is 16 bits and is byte addressable.

1. How many bits are for each tag? How many blocks in the cache?

2. For the following reference stream, indicate whether each reference is a hit or miss: 0, 1, 3, 5, 12, 1, 2, 9, 4

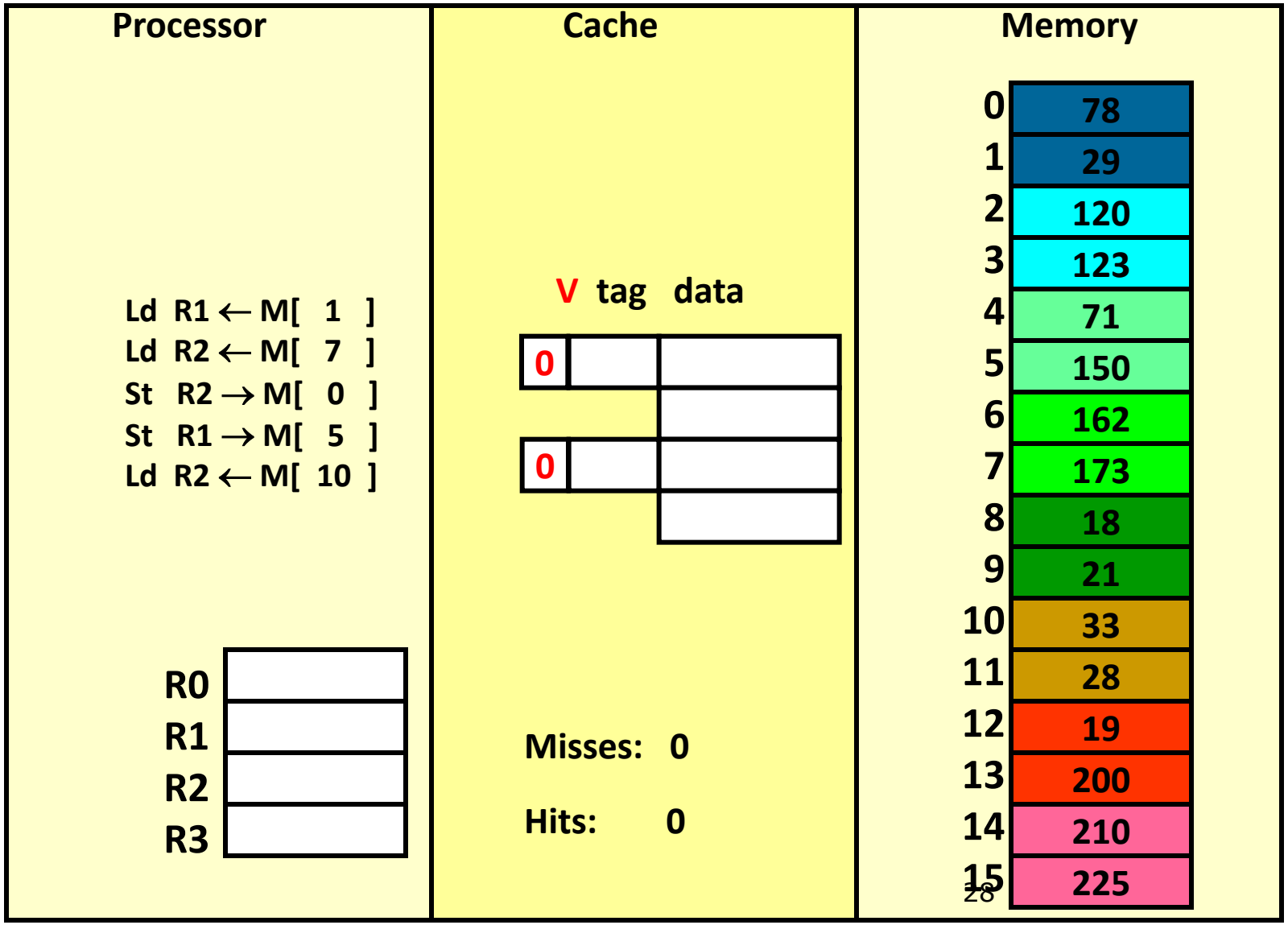
3. What is the hit rate?

4. How many bits are needed for storage overhead?

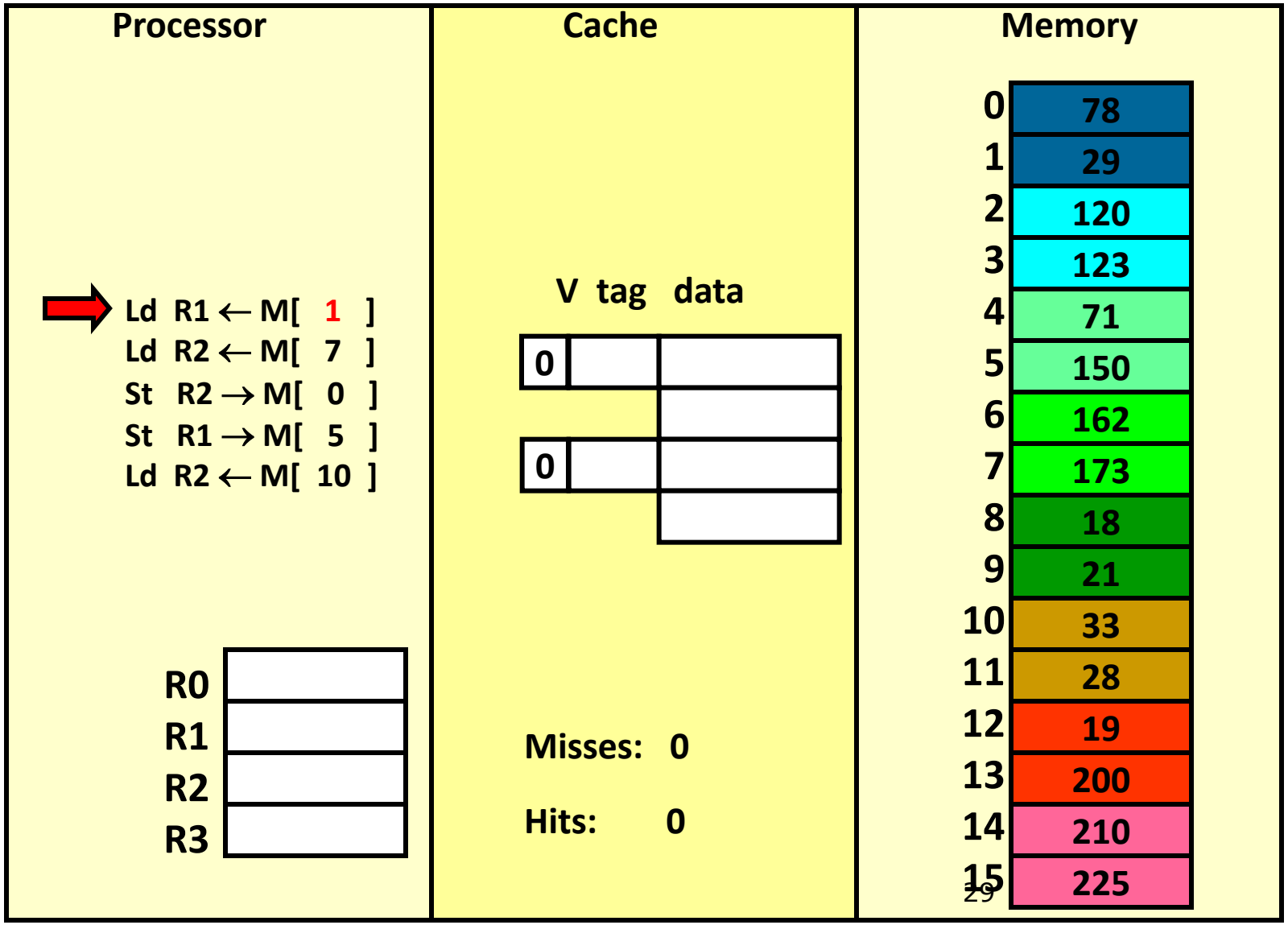
What about stores?

- Where should you write the result of a store?
  - If that memory location is in the cache?
    - Send it to the cache
    - Should we also send it to memory?
      (write-through policy)
  - If it is not in the cache?
    - Allocate the line (put it in the cache)?
      (allocate-on-write policy)
    - Write it directly to memory without allocation?

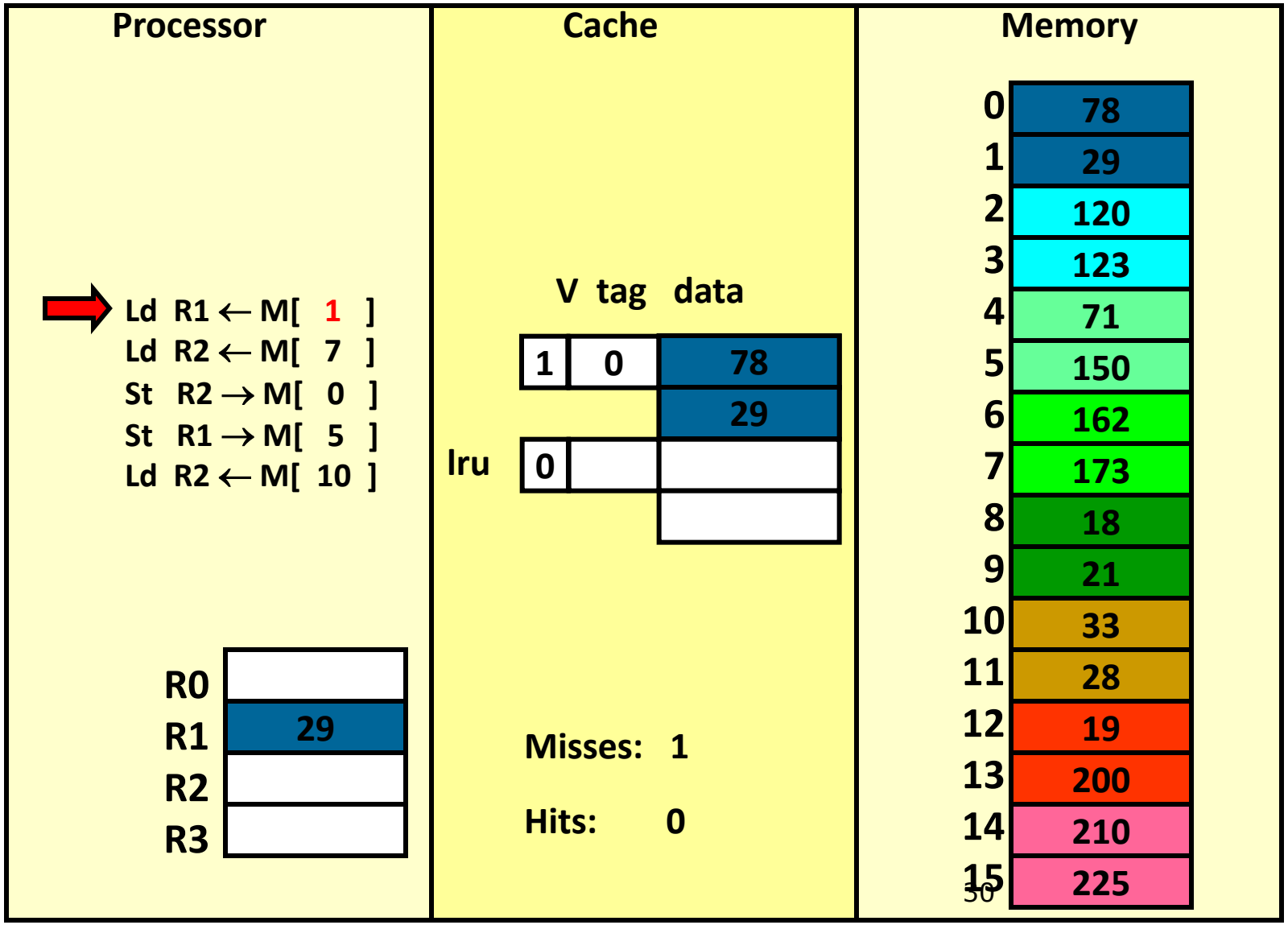
Handling stores (write-through)



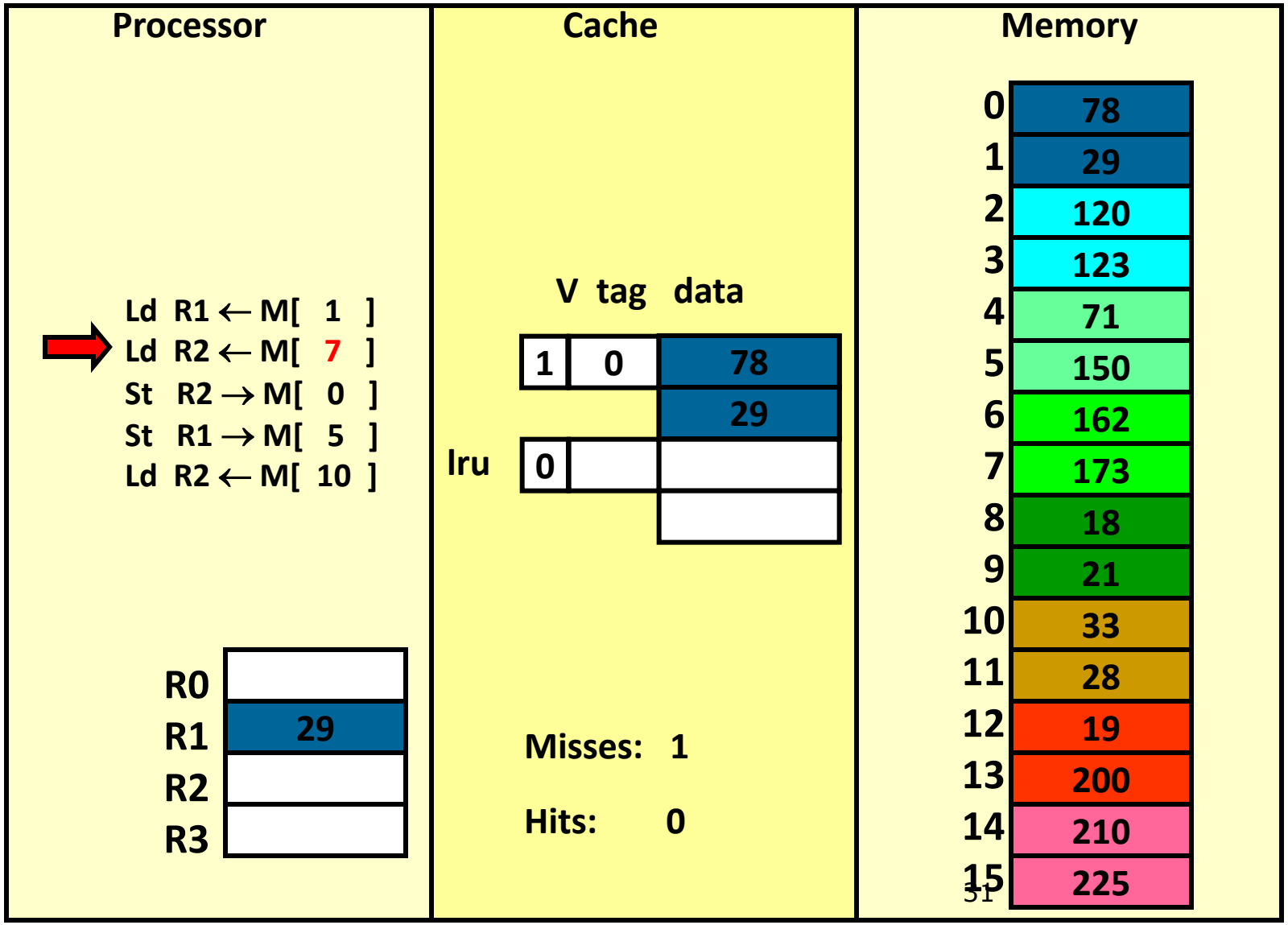
write-through (REF 1)



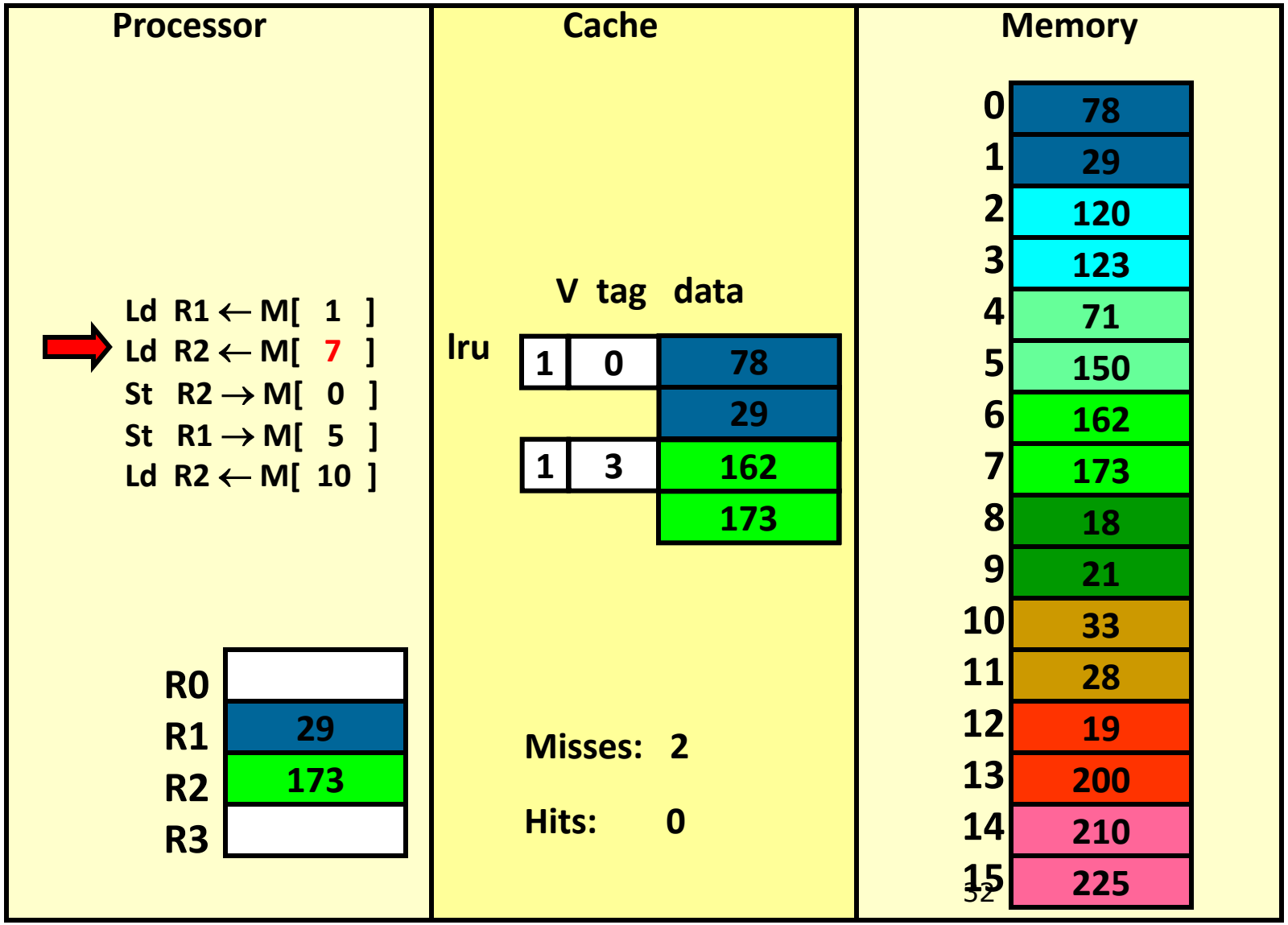
write-through (REF 1)



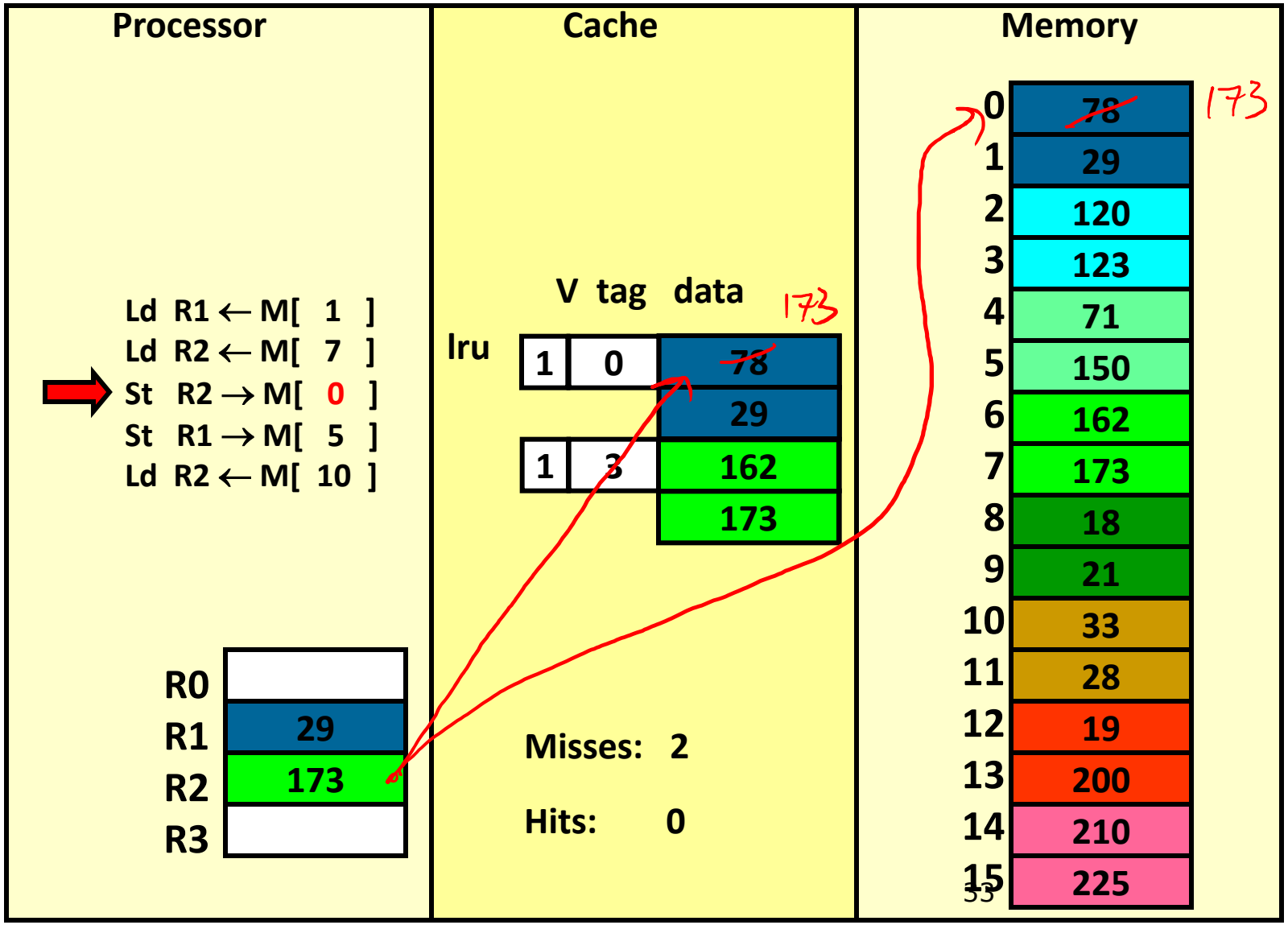
write-through (REF 2)



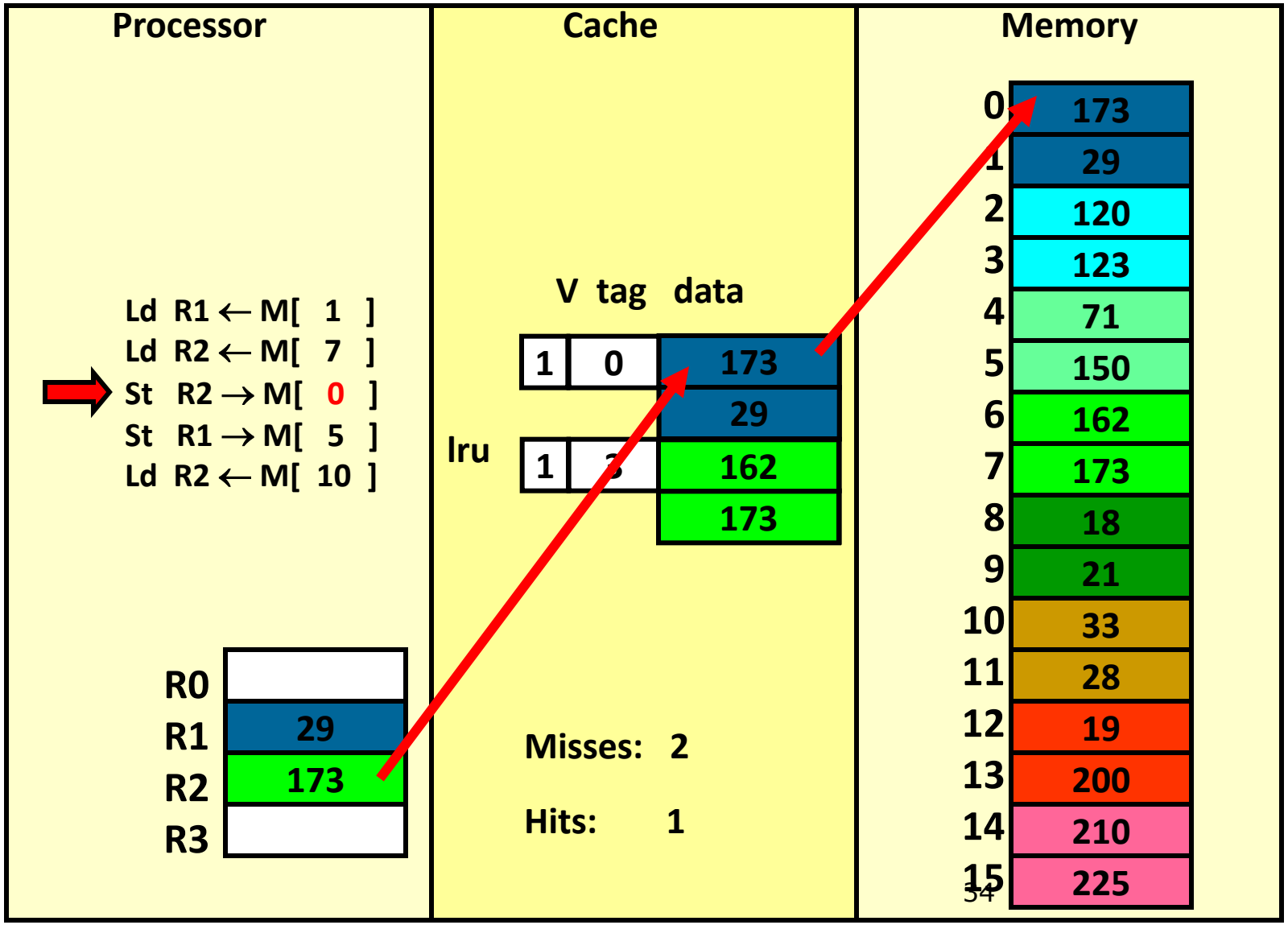
write-through (REF 2)



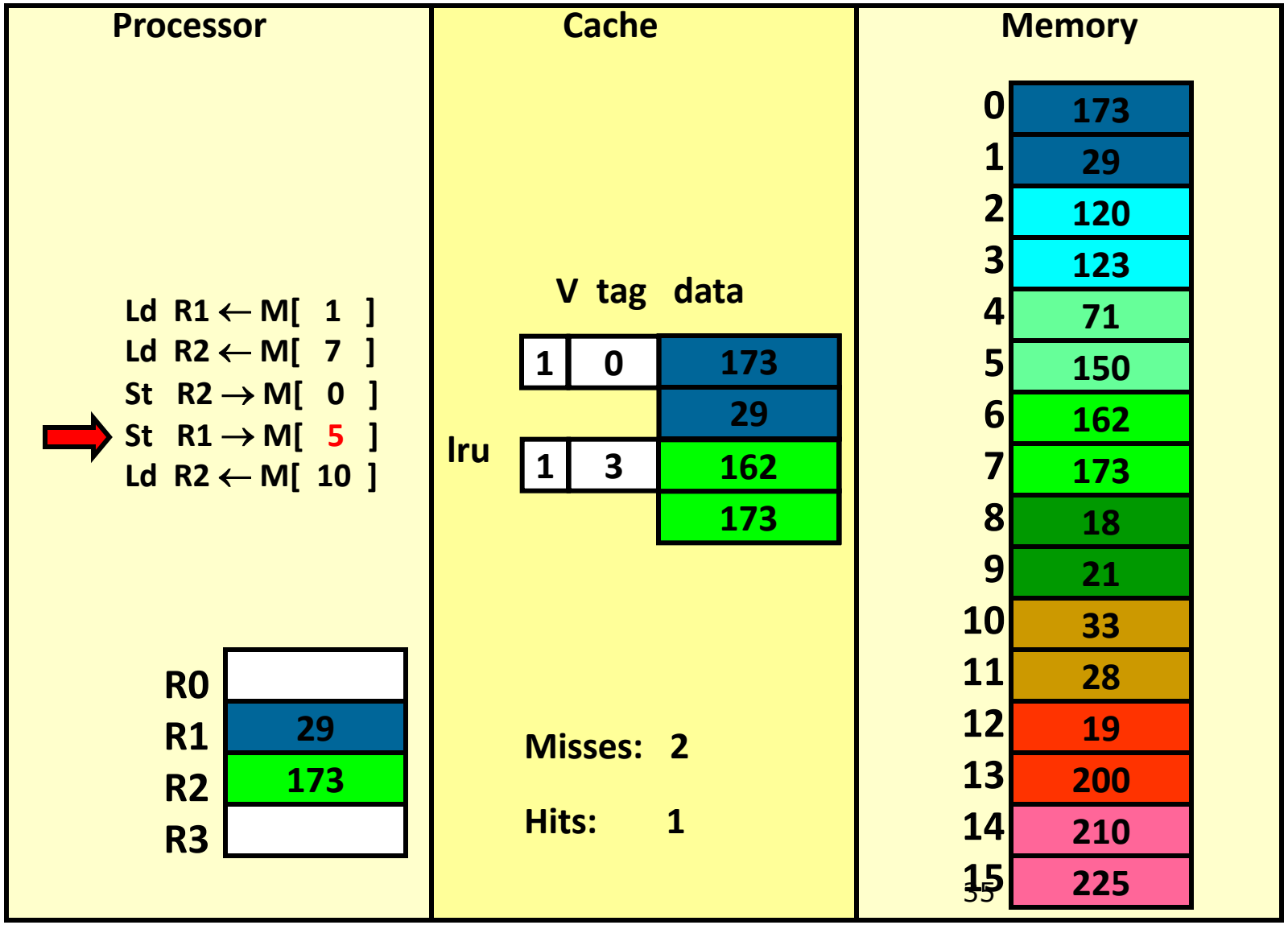
write-through (REF 3)



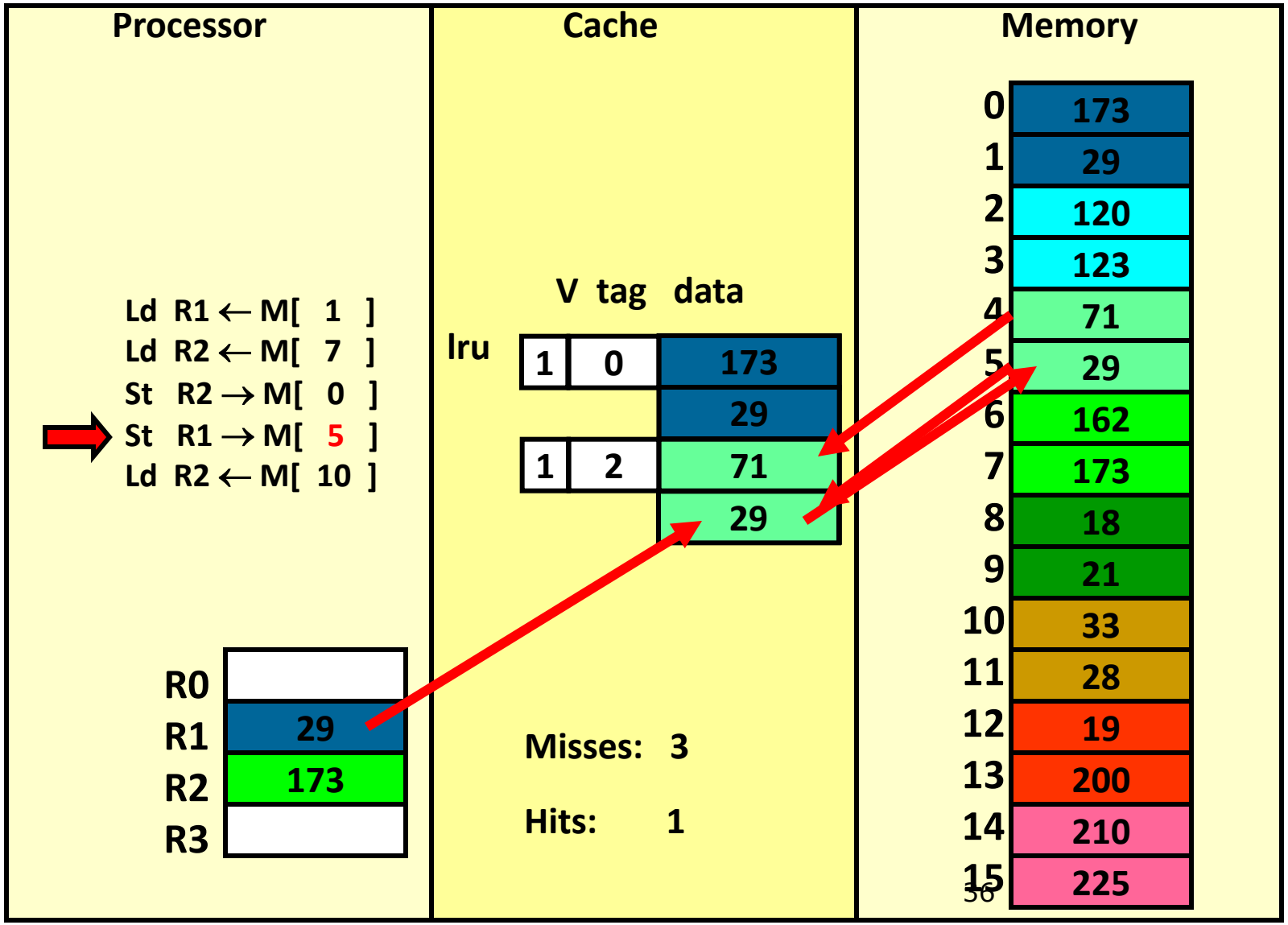
write-through (REF 3)



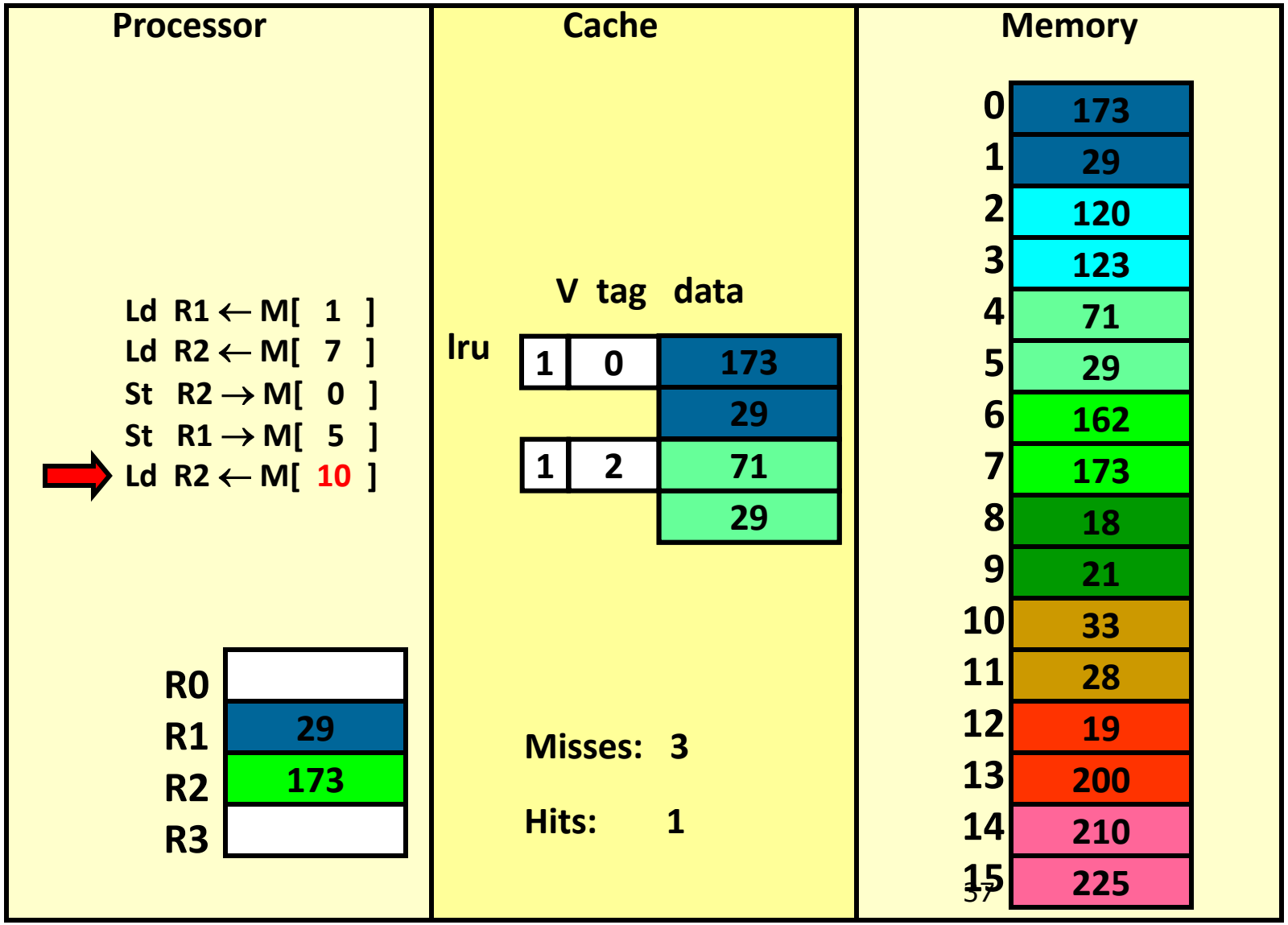
write-through (REF 4)



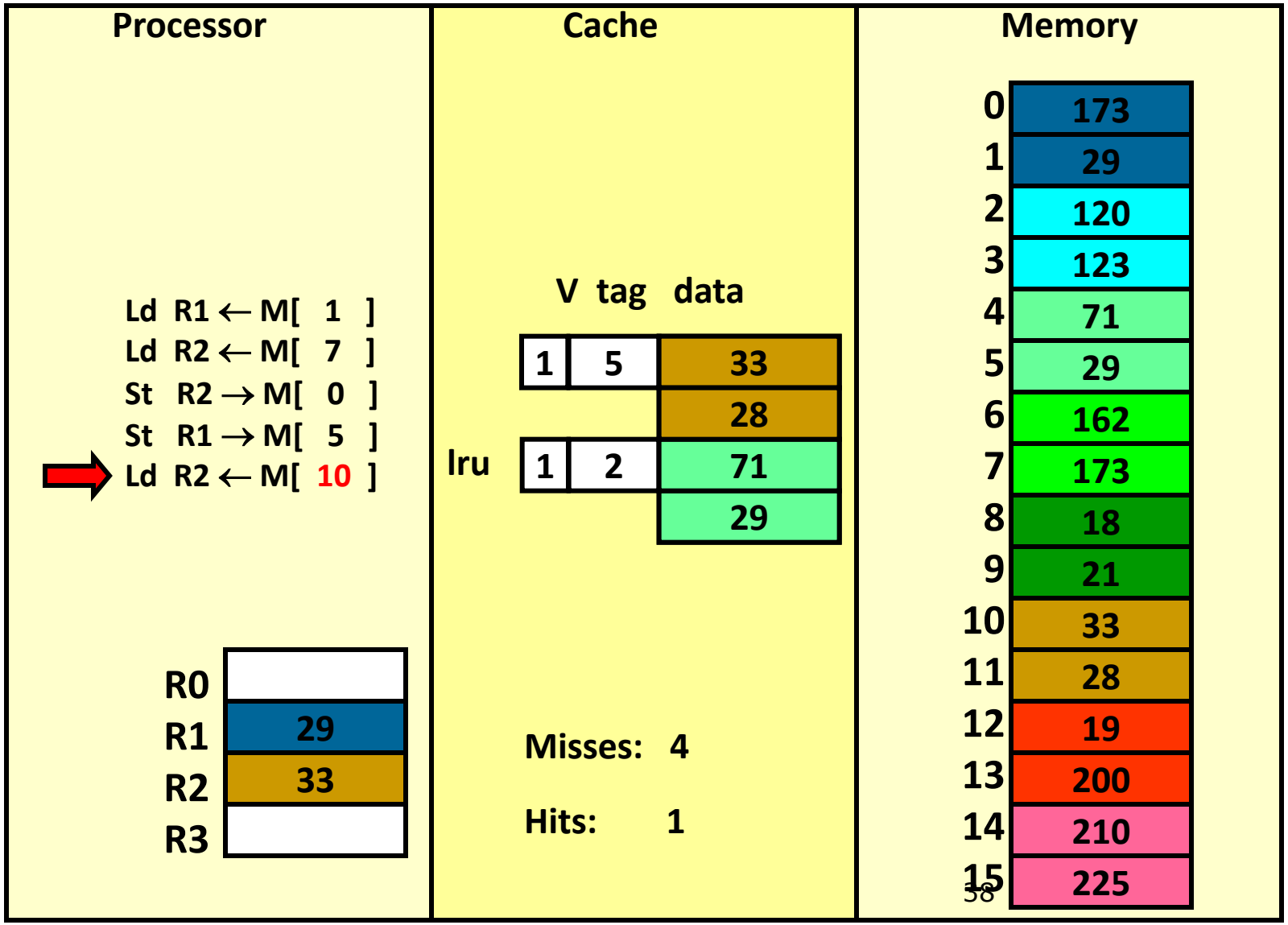
write-through (REF 4)



write-through (REF 5)



write-through (REF 5)



How many memory references?

- Each miss reads a block
  - 2 bytes in this cache
- Each store writes a byte
- Total reads: 8 bytes
- Total writes: 2 bytes

but caches generally miss < 10%

Write-through vs. write-back

- Can we also design the cache to NOT write all stores to memory immediately?
  - We can keep the most recent copy in the cache and update the memory **only when** that data is evicted from the cache (a write-back policy)
  - Do we need to write-back all evicted lines?
    - No, only blocks that have been changed by a store
    - Keep a "dirty bit", reset when the line is allocated, set when the block is stored. If a block is "dirty" when evicted, write its data back to memory

Handling stores (write-back)

Processor

Ld R1 ← M[1]
Ld R2 ← M[7]
St R2 → M[0]
St R1 → M[5]
Ld R2 ← M[10]

R0	
R1	
R2	
R3	

Cache

Valid Dirty

V d	tag	data
0		
0		

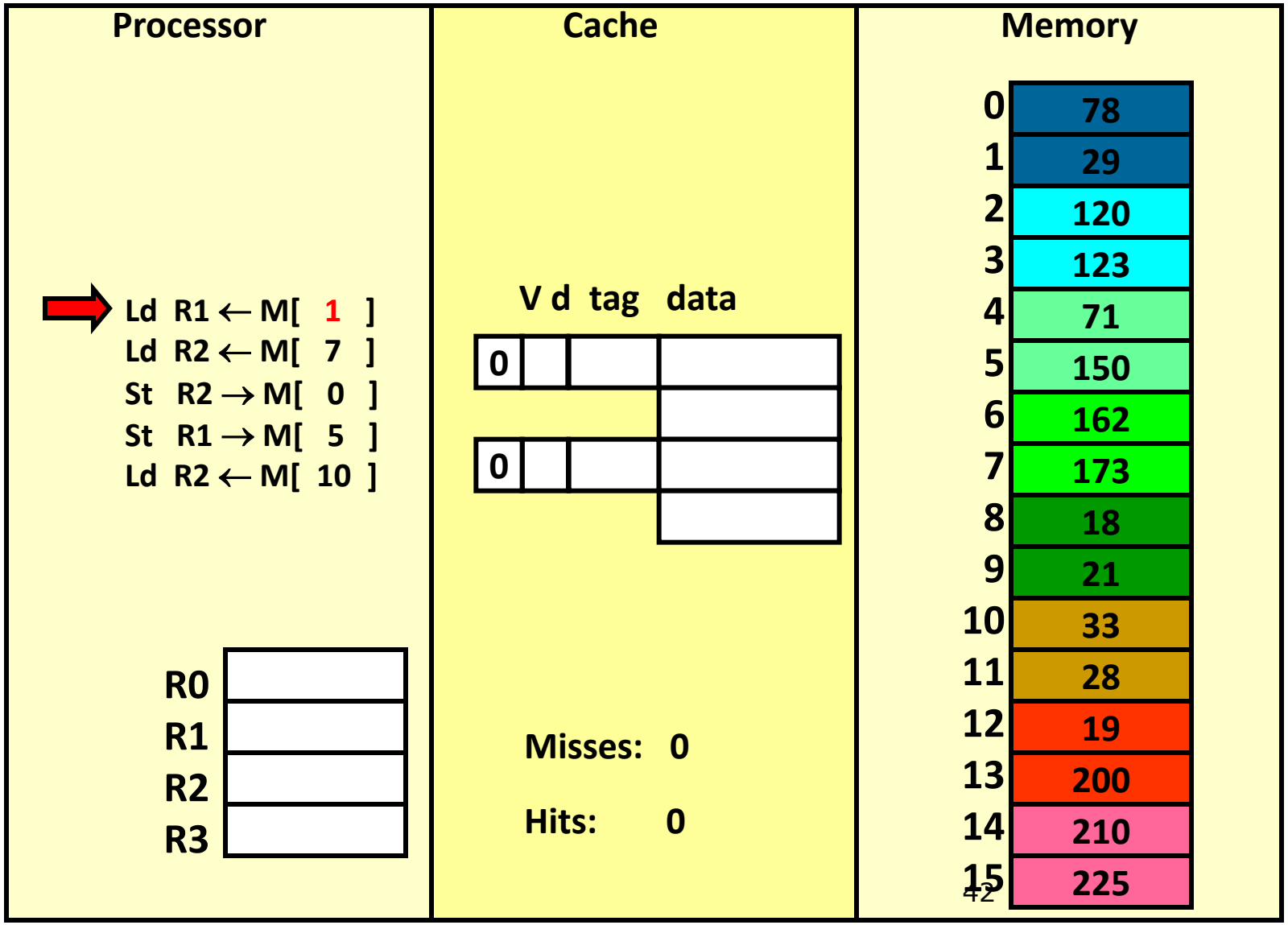
Misses: 0

Hits: 0

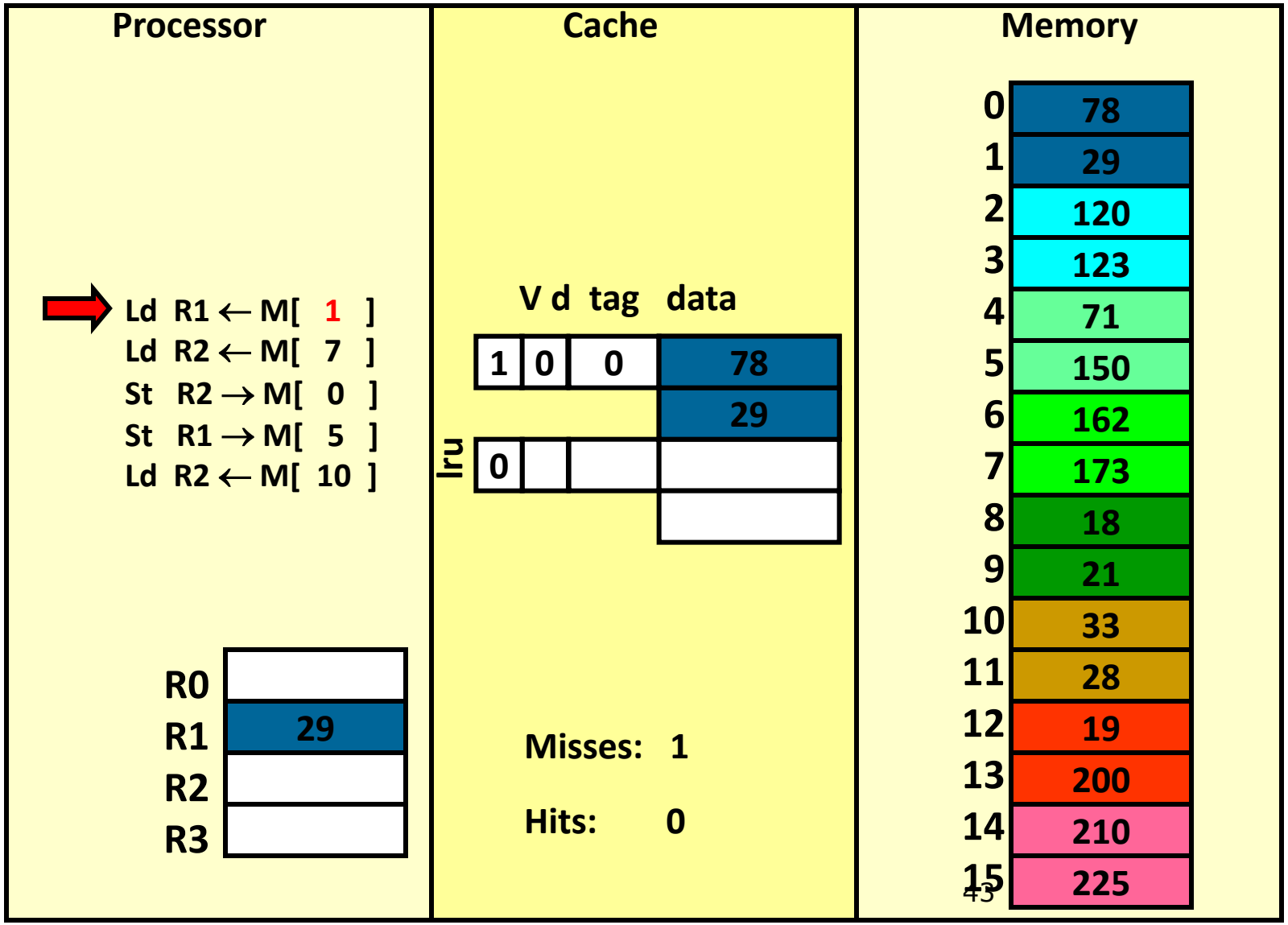
Memory

0	78
1	29
2	120
3	123
4	71
5	150
6	162
7	173
8	18
9	21
10	33
11	28
12	19
13	200
14	210
15	225

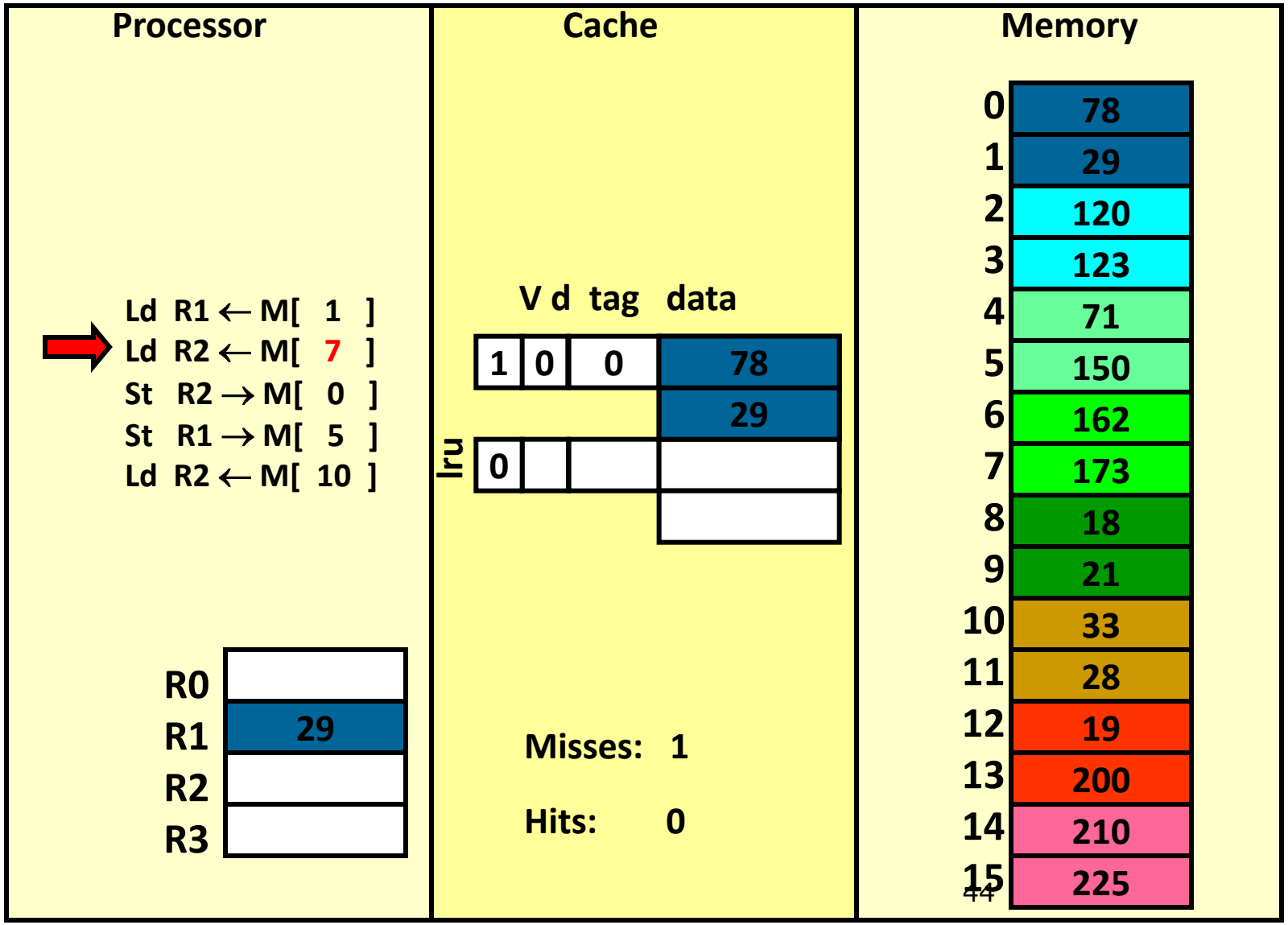
write-back (REF 1)



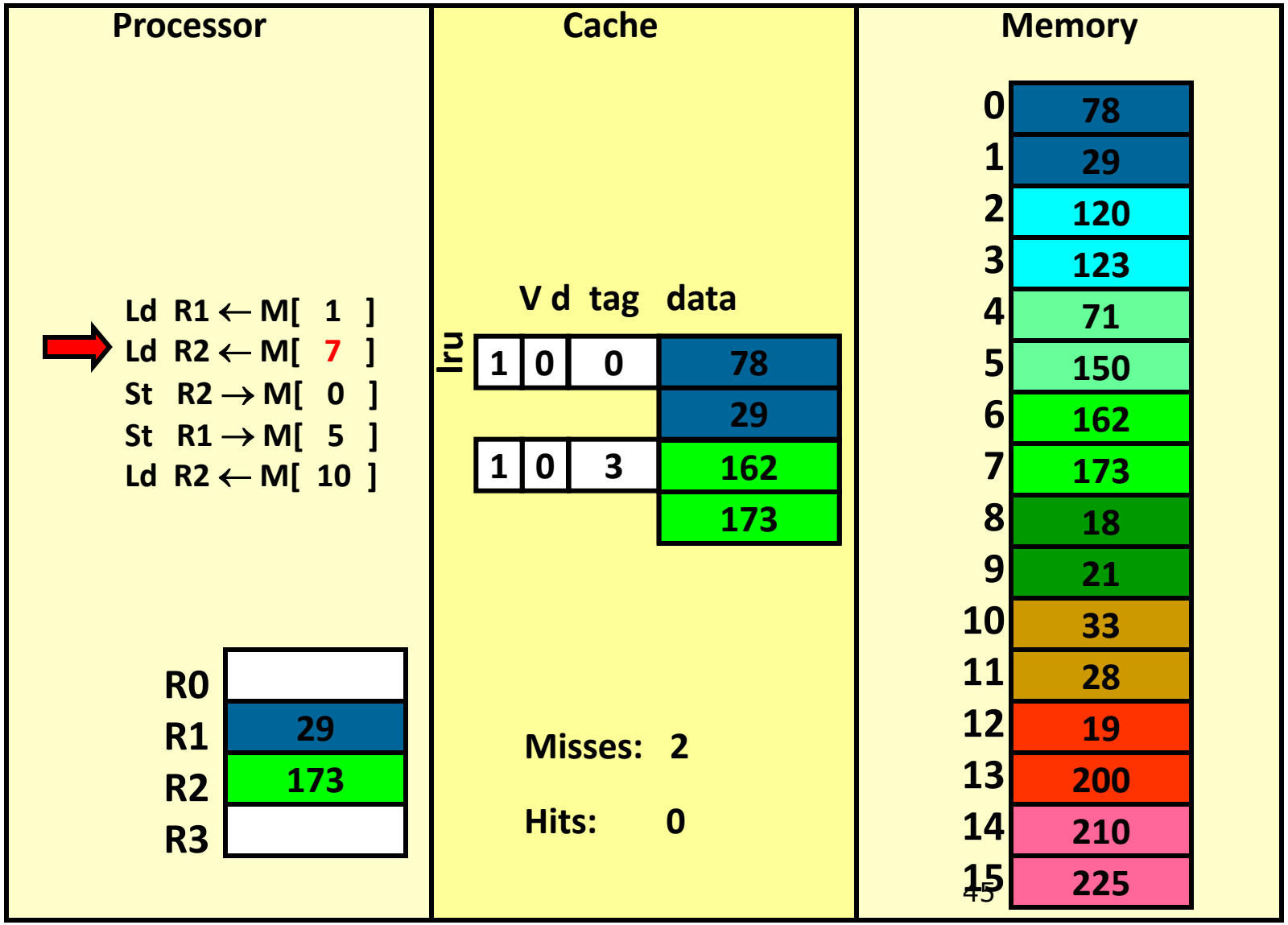
write-back (REF 1)



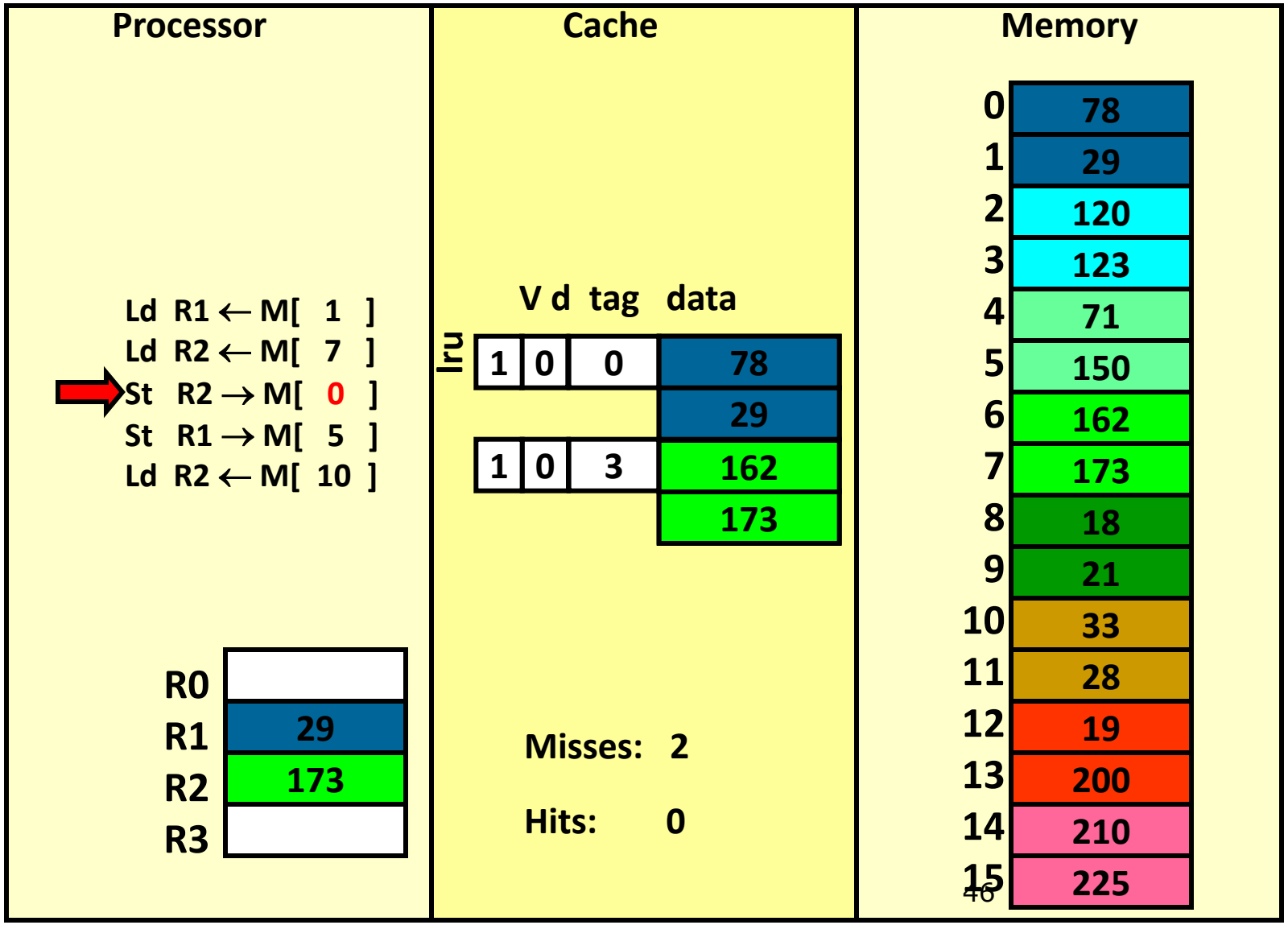
write-back (REF 2)



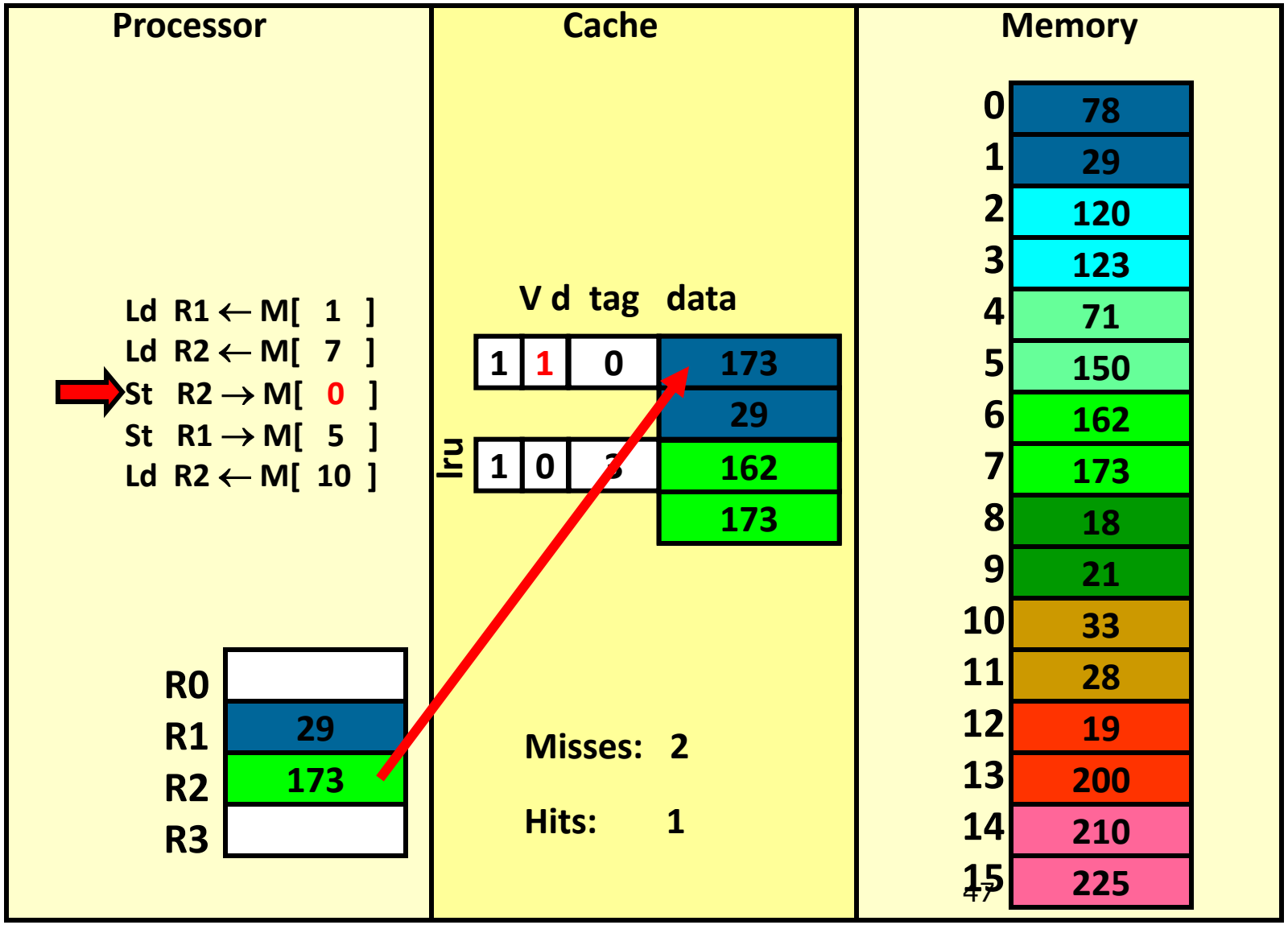
write-back (REF 2)



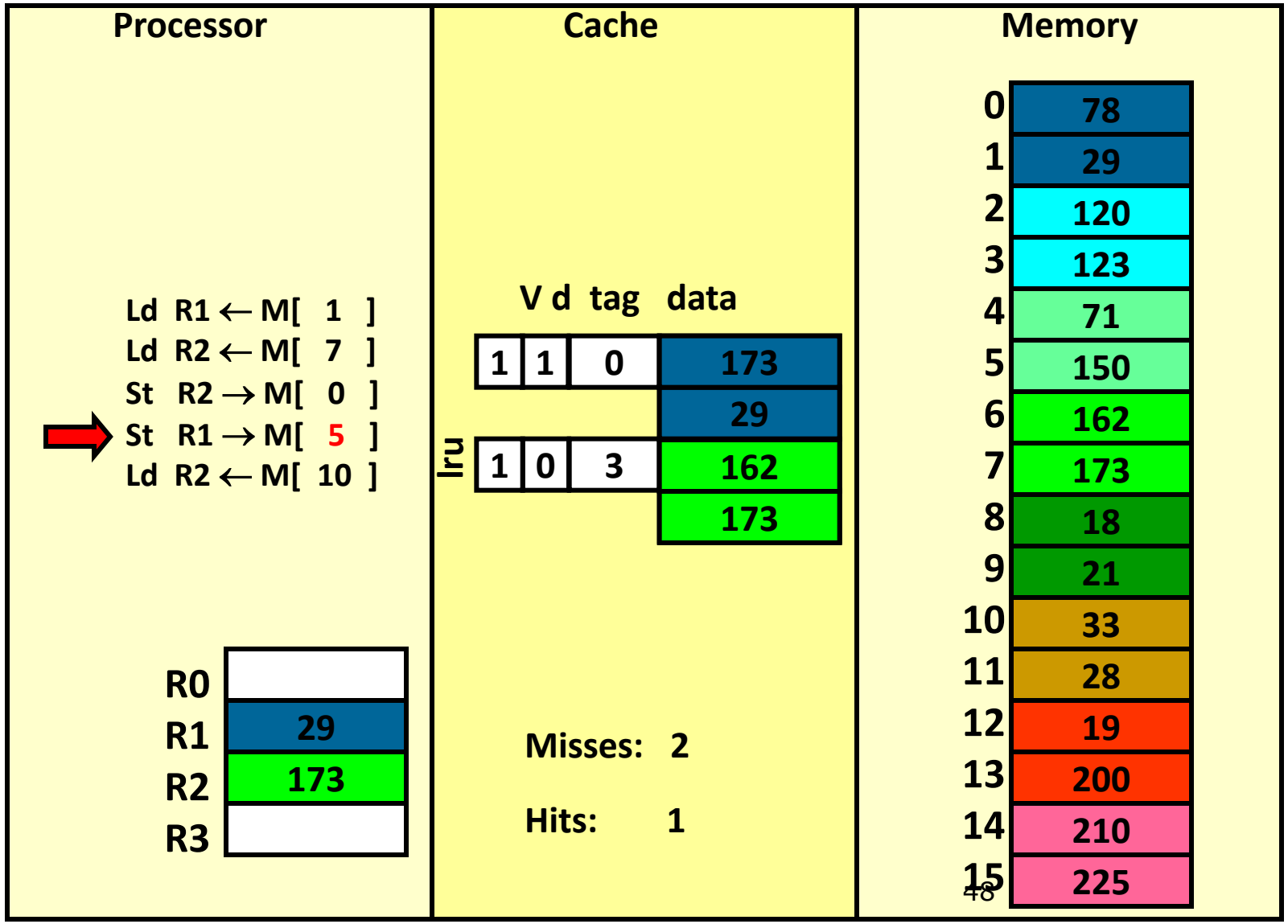
write-back (REF 3)



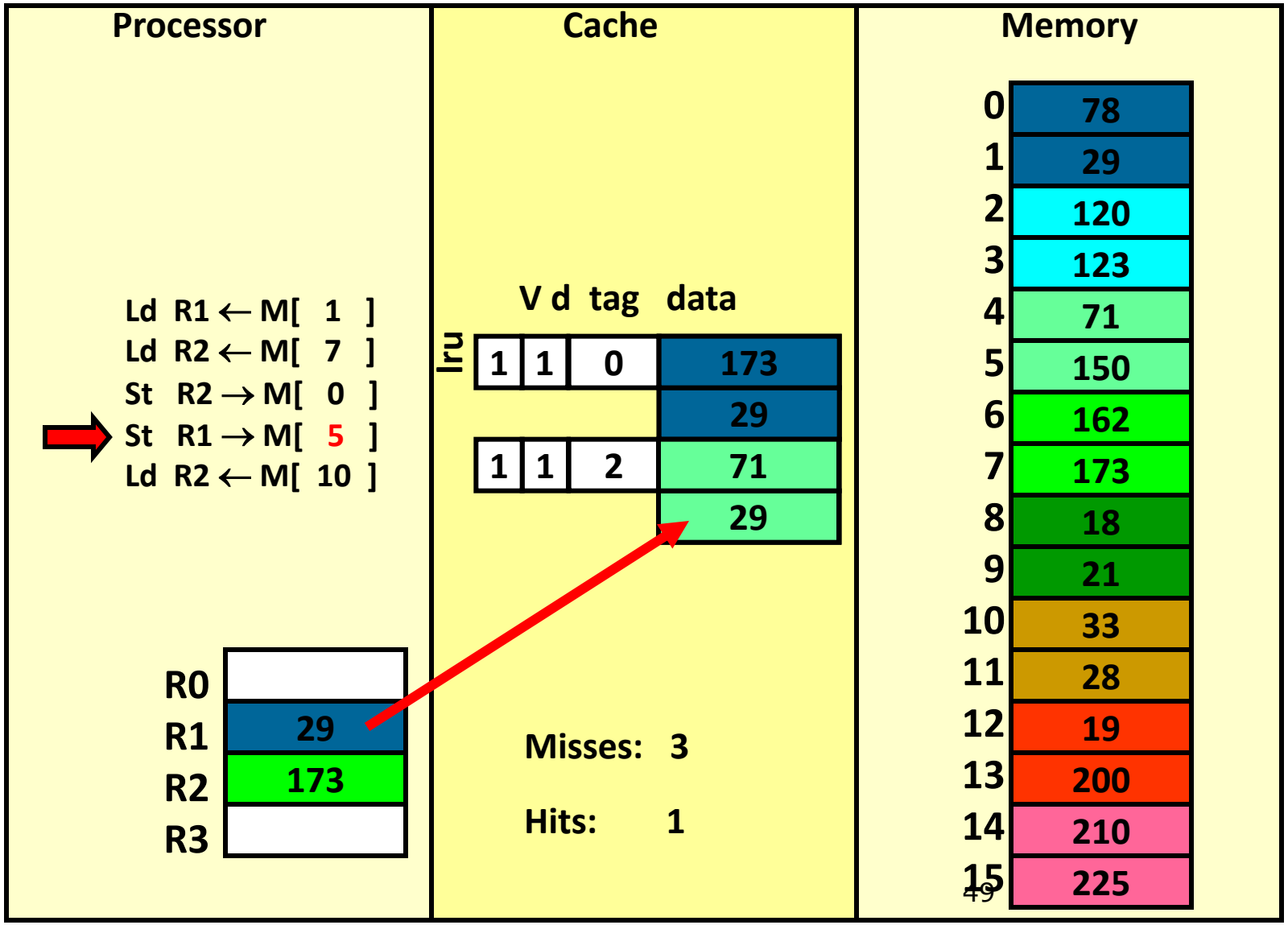
write-back (REF 3)



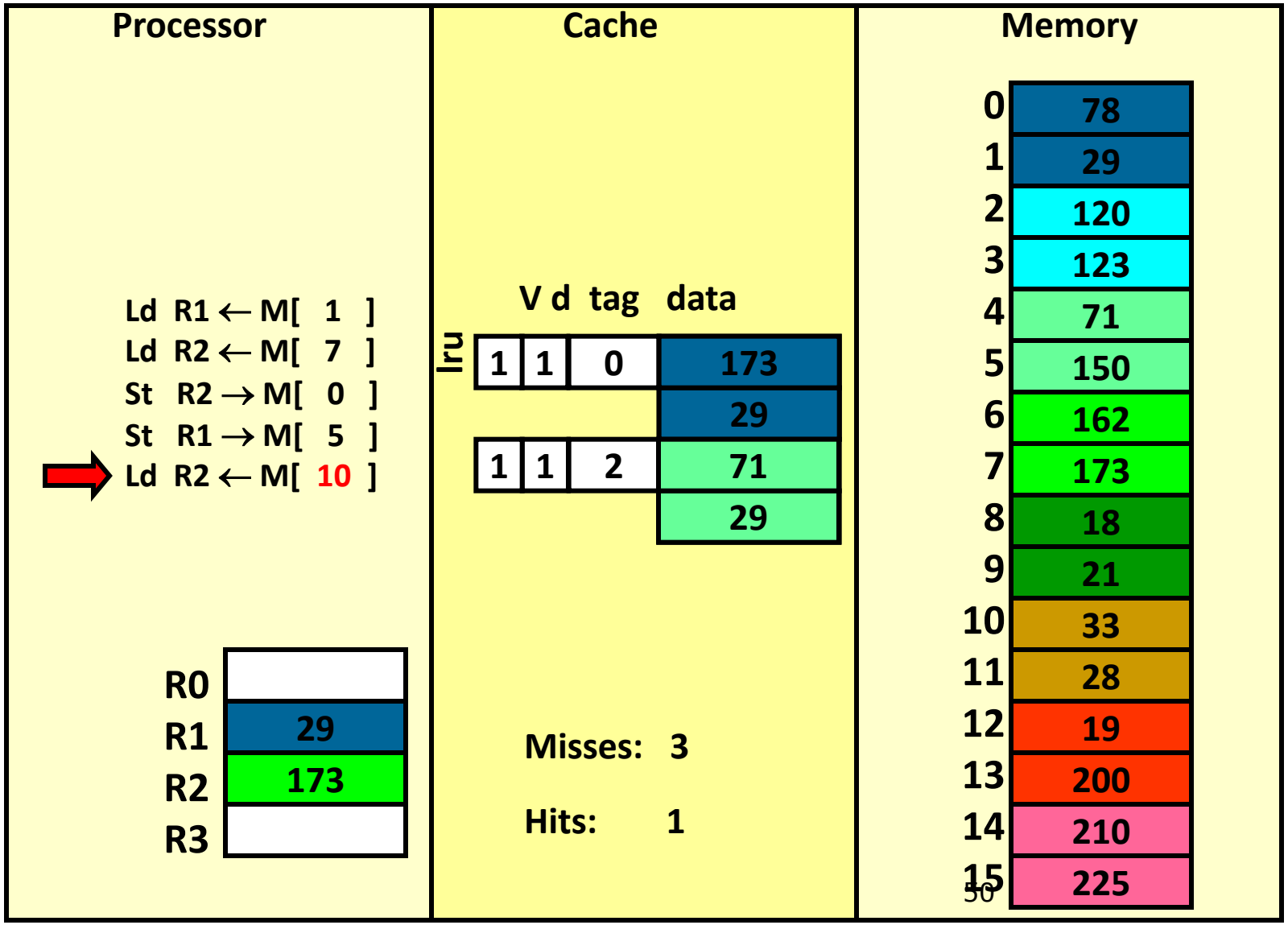
write-back (REF 4)



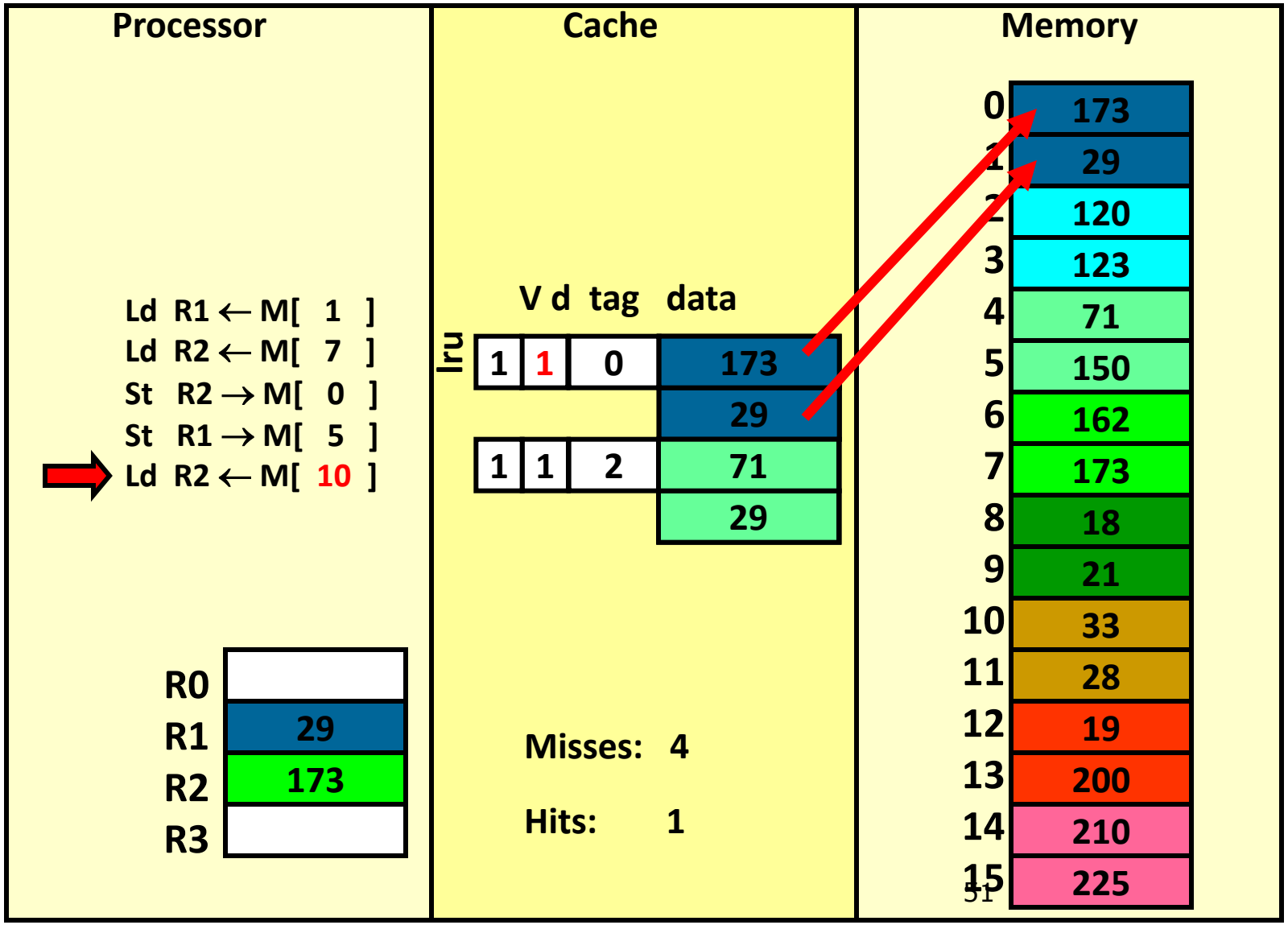
write-back (REF 4)



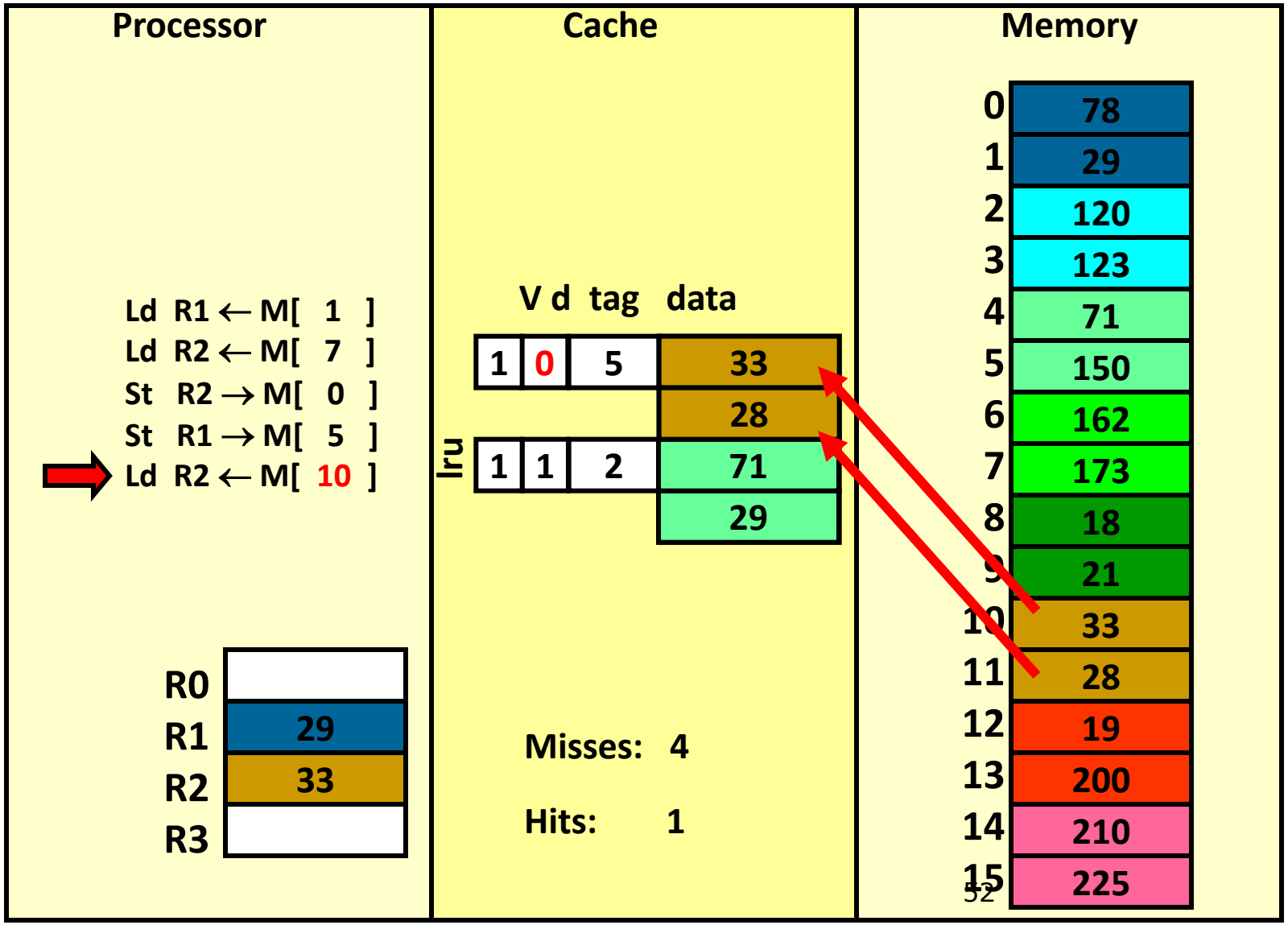
write-back (REF 5)



write-back (REF 5)



write-back (REF 5)



How many memory references?

- Each miss reads a block
  - 2 bytes in this cache
- Each evicted dirty cache line writes a block
- Total reads: 8 bytes
- Total writes: 4 bytes (after final eviction)

For this example, would you choose write-back or write-through?

Class Problem 2

- Consider the following cache:
  32-bit memory addresses, byte addressable, 64KB cache
  64B cache block size, write-allocate, write-back, ~~*fully associative*~~

  This cache will need 512 kilobits for the data area (64 kilobytes times 8 bits per byte). Note that in this context, 1 kilobyte = 1024 bytes (NOT 1000 bytes!) Besides the actual cached data, this cache will need other storage. Consider tags, valid bits, dirty bits, bits to keep track of LRU, and anything else that you think is necessary.

- How many additional bits (not counting the data) will be needed to implement this cache ?

Next Time:

- Virtual Memory

11: Caches II

Engr 315: Hardware / Software Codesign

Andrew Lukefahr
Indiana University